

AI-Powered Sales Forecasting and Business Intelligence Platform for Restaurant Management Using FastAPI, Streamlit, PostgreSQL, and Prophet

TABLE OF CONTENTS

Sl. No	Title	Page No.
	Chapter 1 – Introduction	
1.1	Introduction	
1.2	Company Profile	
1.3	Problem Statement	
1.4	Aim of the Project	
1.5	Objectives	
1.6	Scope of the Project	
1.7	Project Motivation	
1.8	Proposed Solution	
1.9	Project Methodology	
1.10	Organization of the Report	
	Chapter 2 – Literature Survey	
2.1	Introduction	
2.2	Restaurant Management Systems	
2.3	Business Intelligence	
2.4	Point of Sale (POS) Systems	
2.5	Sales Forecasting	
2.6	Time Series Forecasting	
2.7	Facebook Prophet	
2.8	Machine Learning	
2.9	Artificial Intelligence	
2.10	FastAPI	
2.11	Streamlit	
2.12	PostgreSQL	
2.13	SQLAlchemy	
2.14	REST API	
2.15	JWT Authentication	
2.16	Cloud Deployment	
2.17	Comparative Analysis	
2.18	Research Gap	
2.19	Chapter Summary	

2.20	References	
	Chapter 3 – Technologies Used	
3.1	Introduction	
3.2	Python	
3.3	FastAPI	
3.4	Streamlit	
3.5	PostgreSQL	
3.6	SQLAlchemy	
3.7	Pydantic	
3.8	JWT Authentication	
3.9	bcrypt	
3.10	Pandas	
3.11	NumPy	
3.12	Plotly	
3.13	Matplotlib	
3.14	Prophet	
3.15	Git & GitHub	
3.16	Render Cloud Platform	
3.17	Environment Variables	
3.18	Chapter Summary	
	Chapter 4 – System Analysis	
4.1	Introduction	
4.2	Existing System Analysis	
4.3	Limitations of Existing System	
4.4	Proposed System Analysis	
4.5	Advantages of Proposed System	
4.6	Functional Requirements	
4.7	Non-Functional Requirements	
4.8	Feasibility Study	
4.9	Software Requirement Specification	
4.10	User Roles	
4.11	Use Case Analysis	
4.12	Chapter Summary	
	Chapter 5 – System Design	
5.1	Introduction	
5.2	Overall System Architecture	
5.3	Layered Architecture	
5.4	Module Design	
5.5	Authentication Workflow	

5.6	CRUD Workflow	
5.7	Forecast Workflow	
5.8	Dashboard Workflow	
5.9	Component Diagram	
5.10	Deployment Diagram	
5.11	Database Design Overview	
5.12	API Design	
5.13	Security Design	
5.14	Chapter Summary	
	Chapter 6 – Database Design	
6.1	Introduction	
6.2	Database Architecture	
6.3	Database Normalization	
6.4	Users Table	
6.5	Restaurant Bills Table	
6.6	Restaurant Bill Items Table	
6.7	Restaurant Sales Table	
6.8	Audit Logs Table	
6.9	Entity Relationship Diagram	
6.10	Database Constraints	
6.11	Indexing Strategy	
6.12	Data Flow	
6.13	Security Considerations	
6.14	Advantages of Database Design	
6.15	Chapter Summary	
	Chapter 7 – System Implementation	
7.1	Introduction	
7.2	Project Folder Structure	
7.3	Database Configuration	
7.4	FastAPI Backend Implementation	
7.5	User Authentication Module	
7.6	JWT Authentication	
7.7	Role-Based Access Control	
7.8	Sales Management Module	
7.9	Create Bill Implementation	
7.10	Read Bill Implementation	
7.11	Update Bill Implementation	
7.12	Soft Delete Implementation	
7.13	Restore Bill Implementation	
7.14	Audit Logging	

7.15	CRUD API Flow	
7.16	Database Transactions	
7.17	Historical Data Import	
7.18	Data Preprocessing	
7.19	Business Intelligence Dashboard	
7.20	Key Performance Indicators	
7.21	Interactive Data Visualization	
7.22	Forecasting Module	
7.23	Model Monitoring	
7.24	Report Generation	
7.25	AI Assistant Module	
7.26	Version Control Using Git	
7.27	GitHub Repository	
7.28	Cloud Deployment Using Render	
7.29	Environment Variable Configuration	
7.30	Challenges Faced During Development	
7.31	Future Scalability	
7.32	Chapter Summary	
	Chapter 8 – System Testing	
8.1	Introduction	
8.2	Testing Methodology	
8.3	Unit Testing	
8.4	Integration Testing	
8.5	System Testing	
8.6	API Testing	
8.7	Authentication Testing	
8.8	CRUD Testing	
8.9	Dashboard Testing	
8.10	Forecast Testing	
8.11	Historical Data Import Testing	
8.12	Report Generation Testing	
8.13	Deployment Testing	
8.14	User Acceptance Testing	
8.15	Test Case Summary	
8.16	Bug Fixes During Development	
8.17	Testing Summary	
	Chapter 9 – Results and Discussion	
9.1	Introduction	
9.2	Login Module Results	
9.3	Dashboard Results	

9.4	KPI Results	
9.5	Revenue Analysis Results	
9.6	Category Analysis Results	
9.7	Payment Method Analysis Results	
9.8	Historical Data Analysis Results	
9.9	Sales Forecast Results	
9.10	CRUD Operation Results	
9.11	Audit Logging Results	
9.12	Report Generation Results	
9.13	Deployment Results	
9.14	Performance Evaluation	
9.15	Comparison of Objectives and Achievements	
9.16	Overall Discussion	
9.17	Chapter Summary	
	Chapter 10 – Conclusion and Future Enhancements	
10.1	Introduction	
10.2	Project Summary	
10.3	Objectives Achieved	
10.4	Project Outcomes	
10.5	Benefits of the Developed System	
10.6	Limitations of the Current System	
10.7	Future Enhancements	
10.8	Conclusion	
	Appendices	
A	API Documentation	
B	Database Schema	
C	Installation Guide	
D	Deployment Guide	
E	User Manual	
F	Sample Screenshots	
	References	

CHAPTER 1

INTRODUCTION

1.1 Introduction

The restaurant industry has experienced significant digital transformation over the past decade, driven by increasing customer expectations, rapid technological advancements, and the need for data-driven decision-making. Modern restaurants generate a vast amount of operational data every day through billing systems, customer transactions, payment methods, menu sales, delivery platforms, and customer preferences. Although these data sources provide valuable business insights, many restaurants still rely on traditional reporting methods that offer only historical summaries without predictive capabilities.

Sales forecasting has become an essential component of business intelligence because it enables organizations to estimate future sales, optimize inventory, allocate staff efficiently, reduce operational costs, and improve profitability. Accurate forecasting helps restaurant managers anticipate customer demand during peak hours, festivals, weekends, and seasonal variations. Conventional forecasting techniques based on manual calculations or spreadsheet analysis often fail to provide reliable predictions due to the complexity of modern business environments.

Artificial Intelligence (AI), Machine Learning (ML), and Business Intelligence (BI) technologies have transformed the way organizations analyze operational data. By integrating advanced forecasting algorithms with interactive dashboards, organizations can gain meaningful insights into business performance and make informed strategic decisions. Time-series forecasting models such as Prophet have gained popularity because of their ability to capture trends, seasonality, holidays, and business fluctuations with minimal configuration.

This internship project, titled "**AI-Powered Sales Forecasting and Business Intelligence Platform for Restaurant Management Using FastAPI,**

Streamlit, PostgreSQL, and Prophet," focuses on developing an intelligent decision-support system for restaurant management. The system integrates a modern Point-of-Sale (POS) data management module, interactive analytical dashboards, secure authentication mechanisms, and predictive analytics into a single unified platform.

The developed application enables restaurant administrators and managers to manage sales transactions efficiently while simultaneously analyzing historical sales patterns and forecasting future revenue. Interactive dashboards provide real-time Key Performance Indicators (KPIs), revenue trends, category-wise sales analysis, payment method distribution, meal period analysis, and other business metrics. Additionally, the forecasting module predicts future sales using Facebook Prophet, enabling management to prepare for future business demands.

The application follows a modular architecture based on FastAPI for backend services, PostgreSQL for database management, Streamlit for interactive dashboard visualization, SQLAlchemy for database communication, and JWT-based authentication for secure access control. Role-Based Access Control (RBAC) ensures that users with different responsibilities, such as administrators, managers, and cashiers, receive appropriate permissions based on their organizational roles.

The project also incorporates features such as audit logging, soft deletion and restoration of records, historical data analysis using imported CSV datasets, and secure RESTful APIs for seamless communication between different system components. These features collectively enhance business transparency, operational efficiency, and decision-making capabilities.

Overall, the developed system demonstrates how modern software engineering practices combined with Business Intelligence and forecasting technologies can significantly improve restaurant operations while providing a scalable foundation for future AI-based enhancements.

1.2 Problem Statement

Many restaurants continue to rely on conventional Point-of-Sale systems that primarily record billing transactions without providing advanced analytical capabilities. Although these systems successfully manage daily operations, they

often fail to assist business owners in understanding customer behavior, predicting future sales, identifying profitable products, or optimizing operational efficiency.

The primary problems identified in existing restaurant management systems include:

- Lack of accurate sales forecasting mechanisms.
- Limited business intelligence and visualization capabilities.
- Absence of centralized analytical dashboards.
- Manual preparation of business reports.
- No role-based access management.
- Insufficient data security and audit tracking.
- Difficulty in identifying sales trends and customer preferences.
- Limited scalability for future AI integration.

These limitations reduce operational efficiency and prevent management from making proactive business decisions based on data-driven insights.

1.3 Existing System

Traditional restaurant management systems primarily focus on transaction processing rather than strategic decision-making. Most available systems maintain billing records, inventory details, and payment information but provide only basic reports generated from historical data.

Existing systems generally possess the following characteristics:

- Manual report generation.
- Limited visualization capabilities.
- No predictive analytics.
- Basic authentication mechanisms.
- Static sales reports.
- No integrated forecasting models.

- Minimal role-based security.
- Lack of audit logging.
- Limited scalability for future enhancements.

Consequently, restaurant owners are unable to effectively analyze business performance or prepare for future customer demand using intelligent forecasting techniques.

1.4 Limitations of Existing System

The major limitations identified in traditional restaurant management systems include:

- Absence of intelligent forecasting mechanisms.
 - Limited business analytics and interactive visualization.
 - Manual sales analysis using spreadsheets.
 - Inadequate role-based authorization.
 - No comprehensive audit trail.
 - Inability to restore deleted records.
 - Poor integration between operational data and analytical insights.
 - Difficulty in analyzing historical sales trends.
 - Limited scalability for cloud deployment.
 - Lack of AI-assisted business recommendations.
-

1.5 Proposed System

The proposed system is an intelligent, AI-powered Sales Forecasting and Business Intelligence Platform designed specifically for restaurant management. Unlike conventional restaurant management systems that primarily record transactions, the proposed solution integrates operational data management, interactive business analytics, predictive forecasting, secure authentication, and audit logging into a single centralized platform.

The system adopts a modular architecture consisting of a FastAPI backend, PostgreSQL relational database, Streamlit interactive dashboard, and Prophet forecasting model. Each component is designed to perform a specialized function while maintaining seamless communication through RESTful APIs.

The application begins with a secure authentication module where users log into the system using role-based credentials. JSON Web Token (JWT) authentication is implemented to ensure secure communication between the client interface and backend services. Role-Based Access Control (RBAC) restricts user actions based on organizational responsibilities such as Administrator, Manager, and Cashier.

Restaurant sales transactions are recorded through a Point-of-Sale (POS) billing module. The billing system captures customer information, payment methods, order details, tax calculations, service charges, delivery charges, menu items, and revenue information. The data is stored in PostgreSQL using a normalized relational database structure to ensure consistency and efficient querying.

The system automatically transforms billing information into analytical datasets that support business intelligence operations. Historical sales records are maintained in the database and can also be imported from CSV datasets to facilitate long-term trend analysis. This enables restaurant managers to analyze business performance using several interactive dashboards.

The Business Intelligence Dashboard provides comprehensive visualization of restaurant operations through Key Performance Indicators (KPIs), monthly revenue trends, category-wise sales analysis, payment method distribution, meal period analysis, top-selling products, customer traffic, and other performance indicators. Interactive filters allow users to explore business data dynamically across different periods and categories.

To support future planning, the platform integrates Facebook Prophet, a modern time-series forecasting library capable of predicting future sales by learning historical trends, seasonal patterns, holidays, and business cycles. The forecasting module generates projected revenue along with confidence intervals, allowing management to make informed business decisions.

Additional enterprise features such as audit logging, soft deletion, record restoration, secure REST APIs, downloadable reports, and deployment capabilities further enhance system reliability and maintainability. The modular

architecture also provides flexibility for integrating advanced Artificial Intelligence services and cloud-based infrastructure in future versions.

Overall, the proposed system transforms traditional restaurant management into a modern, intelligent, data-driven decision support platform capable of improving operational efficiency, forecasting business performance, and supporting strategic management decisions.

1.6 Objectives of the Project

The primary objective of this project is to design and develop an intelligent Business Intelligence and Sales Forecasting Platform that enables restaurant owners and management teams to monitor operational performance, analyze historical sales, and predict future revenue using modern analytical techniques.

The specific objectives of the project include:

- To develop a secure Point-of-Sale (POS) management system for restaurant billing.
- To implement Role-Based Access Control (RBAC) using JWT authentication.
- To maintain restaurant transaction data using PostgreSQL.
- To develop RESTful APIs using FastAPI for efficient communication between frontend and backend.
- To provide an interactive dashboard for business analytics using Streamlit.
- To visualize Key Performance Indicators (KPIs) including revenue, transactions, customer traffic, and payment methods.
- To analyze historical restaurant sales data imported from CSV datasets.
- To implement sales forecasting using the Prophet time-series forecasting model.
- To generate future revenue predictions with confidence intervals.
- To implement audit logging for monitoring user activities.
- To support soft deletion and restoration of business records.

- To provide downloadable analytical reports for management.
 - To design a scalable architecture suitable for future cloud deployment and AI integration.
-

1.7 Scope of the Project

The scope of this project encompasses the complete lifecycle of restaurant sales management, business analytics, and predictive forecasting. The system is designed to support restaurant owners, managers, and operational staff by providing centralized management of sales information while enabling advanced business intelligence capabilities.

The application supports the following functional areas:

Operational Scope

- Customer billing management.
- Restaurant sales management.
- Menu item management.
- Revenue calculation.
- Tax and service charge computation.
- Multiple payment method support.
- Customer order recording.
- Transaction history management.

Analytical Scope

- Historical sales analysis.
- Revenue trend visualization.
- Product performance analysis.
- Category-wise sales analysis.
- Meal period analysis.
- Payment method distribution.

- Customer traffic analysis.
- Business KPI monitoring.

Forecasting Scope

- Future sales prediction.
- Revenue forecasting.
- Trend analysis.
- Seasonal analysis.
- Business planning support.

Administrative Scope

- User authentication.
- Role-based authorization.
- Audit logging.
- Soft deletion.
- Record restoration.
- Secure API access.

Although the current system focuses on restaurant management, the underlying architecture can be extended to other retail industries including supermarkets, pharmacies, cafés, hotels, and food delivery businesses.

1.8 Project Features

The developed application incorporates several modern software engineering features that improve operational efficiency and business decision-making.

The major features include:

Authentication Module

- Secure JWT Authentication.
- Password encryption using bcrypt.
- Role-Based Access Control.

- Session security.

POS Billing Module

- Bill generation.
- Customer information management.
- Multiple payment methods.
- Tax calculations.
- Service charge calculations.
- Delivery charge management.
- Tip management.
- Automatic revenue computation.

Sales Management

- Create sales records.
- Update sales records.
- Soft delete functionality.
- Restore deleted records.
- Historical record maintenance.

Business Intelligence Dashboard

- Interactive KPI cards.
- Revenue analysis.
- Monthly sales trends.
- Category-wise analysis.
- Payment method analysis.
- Meal period analysis.
- Weather analysis.
- Customer traffic visualization.

Forecasting Module

- Prophet forecasting model.
- Future revenue prediction.
- Confidence intervals.
- Forecast visualization.
- Performance evaluation.

Reporting Module

- PDF report generation.
- Excel export.
- Business summary.
- Analytical insights.

Security Features

- JWT Authentication.
- Password hashing.
- Audit logs.
- User activity tracking.
- Secure REST APIs.

Deployment Features

- GitHub Version Control.
- Render Cloud Deployment.
- Modular Architecture.
- Environment Variable Configuration.

1.9 Advantages of the Proposed System

The developed platform offers significant advantages over conventional restaurant management systems.

Some of the major advantages include:

- Automated business intelligence.
- Accurate sales forecasting.
- Interactive dashboard visualization.
- Secure user authentication.
- Role-based authorization.
- Centralized data management.
- Improved decision-making.
- Historical trend analysis.
- Reduced manual reporting.
- Scalable architecture.
- Modular software design.
- Cloud deployment readiness.
- Enhanced operational transparency.
- Improved business planning.
- Future AI integration capability.

These advantages enable restaurant management to improve operational efficiency while reducing manual effort and enhancing strategic planning.

1.10 Organization of the Report

This report is organized into ten chapters that collectively describe the complete development lifecycle of the proposed system.

Chapter 1 introduces the project, explains the problem statement, existing system limitations, proposed solution, objectives, scope, features, and advantages.

Chapter 2 presents the literature survey by reviewing previous research on restaurant management systems, business intelligence platforms, sales forecasting techniques, and Artificial Intelligence applications.

Chapter 3 provides a comprehensive overview of the technologies used during system development, including FastAPI, Streamlit, PostgreSQL, Prophet, SQLAlchemy, JWT, GitHub, and Render.

Chapter 4 discusses system analysis, functional requirements, non-functional requirements, feasibility studies, and requirement specifications.

Chapter 5 explains the overall system design through architectural diagrams, database design, workflow diagrams, entity relationship diagrams, and software architecture.

Chapter 6 describes the database schema, relational structure, table relationships, constraints, and normalization process used within the application.

Chapter 7 explains the implementation of various modules, including authentication, billing, forecasting, dashboard development, reporting, audit logging, and deployment.

Chapter 8 presents testing methodologies, API validation, system testing, forecasting evaluation, and software quality assurance procedures.

Chapter 9 discusses the results obtained from the developed application, including dashboard outputs, forecasting performance, business insights, and overall system evaluation.

Chapter 10 concludes the project by summarizing the achievements, discussing project limitations, and presenting future enhancement opportunities for continued system development.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

The rapid evolution of Information Technology has significantly transformed the way businesses collect, process, analyze, and utilize data for decision-making. The restaurant industry, in particular, has experienced substantial digital transformation due to increasing customer expectations, online food delivery platforms, digital payment systems, and cloud-based Point-of-Sale (POS) solutions. Modern restaurants generate enormous volumes of transactional data daily, making efficient data management and intelligent analysis essential for sustainable business growth.

Traditionally, restaurant management focused primarily on recording sales transactions and preparing periodic reports manually. However, these conventional systems provide limited analytical capabilities and fail to assist management in predicting future sales trends or optimizing business operations. As competition increases within the hospitality sector, restaurants require intelligent systems capable of transforming historical operational data into actionable business insights.

Business Intelligence (BI), Artificial Intelligence (AI), Machine Learning (ML), and Predictive Analytics have emerged as powerful technologies that enable organizations to extract meaningful information from large datasets. These technologies facilitate trend identification, demand forecasting, customer behavior analysis, and strategic planning, thereby improving operational efficiency and profitability.

Sales forecasting has become one of the most important applications of predictive analytics in the restaurant industry. Accurate forecasting enables restaurant owners to optimize inventory, reduce food wastage, schedule employees efficiently, improve customer satisfaction, and maximize revenue. Time-series forecasting models such as Facebook Prophet have gained considerable popularity due to their ability to model seasonality, trends, holiday effects, and business fluctuations with minimal parameter tuning.

Similarly, cloud computing and modern web technologies have revolutionized software deployment and accessibility. Frameworks such as FastAPI and Streamlit simplify the development of interactive analytical platforms while PostgreSQL provides reliable enterprise-grade data management. RESTful APIs enable seamless communication between different software components, creating scalable and maintainable business applications.

This chapter presents a comprehensive review of the technologies, methodologies, and previous research related to restaurant management systems, business intelligence platforms, sales forecasting techniques, artificial intelligence applications, cloud-based analytics, and predictive modeling. The objective of this literature survey is to understand existing solutions, identify research gaps, and justify the development of the proposed AI-Powered Sales Forecasting and Business Intelligence Platform.

2.2 Restaurant Management Systems

Restaurant Management Systems (RMS) are specialized software solutions designed to automate and streamline daily restaurant operations. These systems integrate multiple business functions, including billing, inventory management, customer relationship management, employee scheduling, order processing, payment management, and financial reporting.

Initially, restaurant management software primarily focused on replacing manual billing processes with computerized Point-of-Sale (POS) systems. These early systems significantly reduced billing errors, improved transaction speed, and simplified financial record management. However, they lacked analytical capabilities and were unable to provide intelligent insights into business performance.

Modern Restaurant Management Systems have evolved into comprehensive enterprise solutions that integrate various operational modules into a centralized platform. These systems capture every customer transaction, store operational data within relational databases, and generate detailed reports for management.

The major functional components of a modern Restaurant Management System include:

- Point-of-Sale Billing
- Customer Management
- Menu Management
- Order Processing
- Kitchen Display Systems
- Inventory Control
- Employee Management
- Financial Reporting
- Tax Management
- Payment Processing
- Business Analytics

Despite these advancements, many commercially available Restaurant Management Systems continue to focus primarily on operational efficiency rather than predictive business intelligence. Most systems generate historical reports but do not provide intelligent forecasting capabilities that assist managers in future planning.

Furthermore, several existing systems offer limited customization, lack cloud-based deployment flexibility, and do not integrate Artificial Intelligence for strategic decision-making. These limitations highlight the need for intelligent restaurant management platforms capable of combining operational management with predictive analytics and business intelligence.

2.3 Business Intelligence Systems

Business Intelligence (BI) refers to the collection of technologies, methodologies, processes, and software applications used to transform raw organizational data into meaningful information for business decision-making. Business Intelligence systems enable organizations to analyze historical data, identify business trends, evaluate operational performance, and support strategic planning through interactive dashboards and visual analytics.

The primary objective of Business Intelligence is to convert large volumes of structured and unstructured data into actionable insights. Unlike traditional reporting systems that simply display historical information, Business Intelligence platforms provide advanced visualization, interactive exploration, and analytical capabilities that support managerial decision-making.

Business Intelligence typically consists of the following components:

Data Collection

Business data is collected from various operational systems, including billing software, customer databases, inventory management systems, online ordering platforms, and financial applications.

Data Storage

Collected data is stored within centralized relational databases or data warehouses where information can be efficiently queried and managed.

Data Processing

Raw transactional data undergoes cleaning, transformation, validation, and aggregation before analytical processing.

Data Analysis

Statistical methods, machine learning algorithms, and business rules analyze historical data to identify patterns, trends, anomalies, and relationships.

Data Visualization

Interactive dashboards display Key Performance Indicators (KPIs), graphs, charts, tables, heatmaps, and analytical summaries that facilitate business understanding.

Business Intelligence provides several advantages for restaurant management, including:

- Revenue monitoring
- Customer purchasing behavior analysis
- Product performance evaluation
- Seasonal sales analysis

- Operational efficiency monitoring
- Employee productivity assessment
- Profitability analysis
- Financial planning

Interactive dashboards significantly improve managerial efficiency by presenting complex business information through intuitive visual representations. Decision-makers can quickly identify performance issues and implement corrective actions based on real-time analytical insights.

The increasing availability of visualization frameworks such as Plotly, Streamlit, and Power BI has made Business Intelligence more accessible to organizations of all sizes. Cloud-based Business Intelligence platforms further enhance accessibility by enabling managers to monitor business performance from any location.

The proposed project incorporates Business Intelligence principles through an interactive Streamlit dashboard that provides comprehensive sales analytics, KPI monitoring, historical trend visualization, payment method analysis, meal period analysis, product performance evaluation, and forecasting capabilities.

2.4 Point-of-Sale (POS) Systems

A Point-of-Sale (POS) system is the central component of restaurant operations responsible for recording customer transactions, processing payments, generating bills, and maintaining financial records. Modern POS systems have evolved from simple cash registers into integrated digital platforms that support multiple business functions.

The primary responsibilities of a POS system include:

- Customer billing
- Order management
- Menu item selection
- Payment processing
- Tax calculation

- Discount management
- Service charge calculation
- Receipt generation
- Customer data recording
- Daily sales reporting

Restaurants generate thousands of transactions daily, making POS systems the primary source of operational data. Every completed transaction contains valuable information regarding customer preferences, popular menu items, payment methods, seasonal demand, and revenue generation.

These transactional datasets form the foundation for Business Intelligence and Sales Forecasting applications. By analyzing historical POS data, organizations can identify purchasing patterns, optimize menu pricing, improve inventory planning, and forecast future sales demand.

Modern cloud-based POS systems further support integration with online ordering platforms, delivery applications, customer loyalty programs, and third-party payment gateways. Such integrations enhance operational efficiency while providing comprehensive datasets for predictive analytics.

The proposed system extends traditional POS functionality by integrating analytical dashboards, historical sales analysis, forecasting models, audit logging, and secure API-based communication within a unified software platform.

2.5 Sales Forecasting

Sales Forecasting is the process of estimating future sales based on historical business data, market conditions, seasonal trends, promotional activities, and customer purchasing behavior. It plays a critical role in strategic planning by enabling organizations to anticipate future demand and allocate resources effectively.

Accurate sales forecasting provides numerous business benefits, including:

- Inventory optimization
- Reduction of food wastage

- Workforce planning
- Revenue estimation
- Budget preparation
- Demand prediction
- Supply chain optimization
- Business growth planning

Traditional sales forecasting methods often rely on manual calculations or spreadsheet analysis, which may fail to capture complex seasonal patterns and long-term trends. These approaches become increasingly inaccurate as business operations expand and data volume increases.

Machine Learning and Time-Series Forecasting models have significantly improved forecasting accuracy by automatically identifying relationships within historical data. These models learn from previous business behavior and generate future predictions using statistical techniques.

Restaurant sales forecasting presents unique challenges due to fluctuating customer demand, seasonal variations, festivals, weekends, weather conditions, promotions, and changing consumer preferences. Therefore, advanced forecasting algorithms capable of modeling these variations are essential for achieving reliable predictions.

The proposed system addresses these challenges by implementing Facebook Prophet, an advanced time-series forecasting algorithm specifically designed to handle business datasets containing seasonal patterns, trend changes, and holiday effects.

2.6 Time Series Forecasting

Time Series Forecasting is a statistical and machine learning technique used to predict future values based on previously observed chronological data. Unlike conventional prediction models, time series forecasting considers the sequential nature of data and attempts to identify temporal relationships such as trends, seasonality, cyclic behavior, and random variations.

Time series forecasting has become one of the most widely adopted predictive analytics techniques in finance, healthcare, retail, manufacturing, transportation, weather prediction, and restaurant management. Since restaurant sales occur continuously over time, they naturally form time series datasets suitable for forecasting analysis.

A time series consists of observations recorded at regular time intervals, such as daily, weekly, monthly, or yearly sales. Forecasting models analyze these historical observations to estimate future business performance. In restaurant operations, daily sales records, customer visits, revenue generation, order volumes, and menu item demand all represent valuable time series datasets.

The major components of time series data include:

Trend

Trend represents the long-term increase or decrease observed within the dataset. In restaurant businesses, increasing customer demand over several years indicates a positive trend, whereas declining sales indicate a negative trend.

Seasonality

Seasonality refers to repeating patterns that occur over fixed time intervals. Restaurants often experience seasonal variations during weekends, public holidays, festivals, vacation periods, and special events.

Cyclical Patterns

Cyclical behavior occurs over irregular intervals and is generally associated with economic conditions, customer behavior, or industry-wide changes.

Random Variations

Random variations represent unpredictable fluctuations caused by unexpected events such as extreme weather conditions, economic disruptions, or sudden customer behavior changes.

Accurate forecasting requires algorithms capable of identifying all these components simultaneously. Modern forecasting models employ statistical learning techniques to automatically capture these patterns and generate reliable future predictions.

Several forecasting techniques have been developed over the years, including:

- Moving Average
- Exponential Smoothing
- ARIMA
- SARIMA
- Holt-Winters Method
- LSTM Neural Networks
- Facebook Prophet

Among these methods, Prophet has gained significant popularity because of its simplicity, robustness, and ability to handle missing values, trend changes, and multiple seasonal patterns with minimal parameter tuning.

2.7 Facebook Prophet Algorithm

Facebook Prophet is an open-source time series forecasting framework developed by Meta (formerly Facebook). It was specifically designed to simplify business forecasting problems involving strong seasonal effects, trend variations, holidays, and missing observations.

Unlike traditional forecasting models that require extensive statistical expertise, Prophet provides an automated forecasting solution capable of generating highly accurate predictions using only historical timestamped data.

The mathematical model used by Prophet can be expressed as:

$$\text{Forecast} = \text{Trend} + \text{Seasonality} + \text{Holidays} + \text{Error}$$

Each component contributes to the overall prediction process.

Trend Component

The trend component models long-term business growth or decline. Prophet automatically detects trend changes and adapts to changing business conditions using piecewise linear or logistic growth models.

Seasonal Component

Restaurant businesses exhibit multiple seasonal behaviors, including:

- Daily patterns
- Weekly patterns
- Monthly variations
- Annual trends

Prophet models these recurring seasonal effects using Fourier series, allowing accurate representation of periodic business fluctuations.

Holiday Component

Public holidays significantly influence restaurant revenue. Prophet enables users to explicitly define holidays and special events so that forecasting accuracy improves during irregular business periods.

Examples include:

- Christmas
- New Year
- Diwali
- Pongal
- Ramadan
- Independence Day

Error Component

Random business fluctuations that cannot be explained through trends or seasonality are captured within the error component.

Advantages of Prophet

Facebook Prophet provides several advantages over conventional forecasting techniques.

- Simple implementation
- High forecasting accuracy
- Automatic seasonality detection

- Handles missing data
 - Robust against outliers
 - Supports holiday effects
 - Fast model training
 - Minimal parameter tuning
 - Easy visualization
 - Business-oriented forecasting
-

Limitations of Prophet

Despite its advantages, Prophet has several limitations.

- Limited performance for extremely complex nonlinear datasets.
 - Requires sufficient historical observations.
 - Cannot automatically explain business anomalies.
 - Forecast quality depends on data quality.
 - Less suitable for high-frequency streaming data.
-

Prophet in the Proposed System

The developed application utilizes Prophet to forecast future restaurant revenue based on historical transaction records imported into PostgreSQL. Historical sales data undergo preprocessing before model training. Prophet generates future predictions along with confidence intervals, enabling restaurant managers to estimate expected revenue while considering uncertainty.

Interactive visualization of forecast results allows users to compare historical sales with future projections, facilitating better strategic planning.

2.8 Machine Learning in Sales Forecasting

Machine Learning has become one of the most influential technologies in predictive analytics. Unlike traditional statistical methods, Machine Learning

algorithms automatically identify hidden relationships within historical datasets and continuously improve prediction accuracy as additional data becomes available.

Sales forecasting represents one of the most common applications of Machine Learning within Business Intelligence systems. Machine learning models analyze historical sales records, customer behavior, pricing information, promotional campaigns, weather conditions, and seasonal variations to estimate future business performance.

Common Machine Learning algorithms used for sales forecasting include:

- Linear Regression
- Decision Trees
- Random Forest
- Support Vector Machines
- Gradient Boosting
- XGBoost
- Artificial Neural Networks
- Long Short-Term Memory Networks (LSTM)
- Prophet

Each algorithm possesses unique advantages depending on dataset characteristics. Deep Learning techniques such as LSTM perform exceptionally well for highly nonlinear sequential data but require extensive computational resources and large training datasets.

Prophet provides an excellent balance between simplicity and forecasting performance, making it particularly suitable for small and medium-sized businesses that require accurate predictions without extensive machine learning expertise.

Machine Learning-based forecasting provides numerous business advantages:

- Improved prediction accuracy
- Automatic pattern recognition

- Better inventory planning
- Workforce optimization
- Financial planning
- Demand estimation
- Business growth analysis
- Customer behavior prediction

The proposed project integrates Machine Learning through Prophet to generate future revenue forecasts while maintaining simplicity and interpretability.

2.9 Artificial Intelligence in Business Analytics

Artificial Intelligence (AI) has revolutionized Business Analytics by enabling organizations to process large volumes of structured and unstructured data automatically. AI systems can identify hidden relationships, detect anomalies, generate predictions, and provide intelligent recommendations that support managerial decision-making.

Business Analytics traditionally relied on descriptive reporting that explained what had happened. AI extends this capability by introducing predictive and prescriptive analytics that answer questions such as:

- What is likely to happen?
- Why will it happen?
- What actions should management take?

Modern AI-powered Business Intelligence systems incorporate:

- Machine Learning
- Natural Language Processing
- Predictive Analytics
- Recommendation Systems
- Anomaly Detection
- Conversational AI

Within the restaurant industry, AI enables organizations to:

- Forecast sales.
- Predict customer demand.
- Identify best-selling products.
- Detect abnormal business activities.
- Optimize pricing strategies.
- Reduce food wastage.
- Improve customer satisfaction.
- Support management decision-making.

Although the current project temporarily disables cloud-based Large Language Model (LLM) integration for deployment simplicity, the system architecture has been designed to support future AI assistants using services such as Google Gemini or OpenAI. This extensible design allows natural language business queries, automated insight generation, and conversational analytics to be incorporated in future versions.

2.10 Predictive Analytics in Restaurant Management

Predictive Analytics combines statistical techniques, machine learning algorithms, and historical data to estimate future business outcomes. Within restaurant management, predictive analytics has become increasingly valuable for improving operational efficiency and profitability.

Restaurant managers utilize predictive analytics for:

- Revenue forecasting
- Inventory planning
- Employee scheduling
- Menu optimization
- Customer demand estimation
- Marketing campaign planning

- Seasonal business preparation
- Cost reduction

For example, forecasting increased weekend customer demand allows management to increase staffing levels and inventory in advance, thereby reducing service delays and improving customer satisfaction.

Similarly, forecasting low-demand periods helps reduce operational expenses by optimizing employee schedules and minimizing inventory purchases.

The proposed platform integrates predictive analytics directly into the Business Intelligence dashboard. Historical restaurant sales stored within PostgreSQL are analyzed using Prophet to generate interactive forecasts. Management can visualize expected future revenue, compare historical performance with projected sales, and make informed business decisions based on analytical evidence rather than intuition.

By integrating predictive analytics with interactive visualization, the developed system provides a comprehensive decision-support platform that extends beyond traditional reporting and enables proactive restaurant management.

2.11 FastAPI Framework

FastAPI is a modern, high-performance web framework for building RESTful APIs using Python. Developed by Sebastián Ramírez in 2018, FastAPI has rapidly become one of the most popular backend development frameworks because of its exceptional speed, automatic API documentation, asynchronous programming support, and built-in data validation.

FastAPI is built on top of **Starlette** for web handling and **Pydantic** for data validation. These technologies enable developers to create secure, scalable, and efficient backend services with minimal code. FastAPI also automatically generates interactive API documentation using Swagger UI and ReDoc, allowing developers to test endpoints without requiring third-party software.

One of the major advantages of FastAPI is its support for asynchronous request handling. Asynchronous programming allows the server to process multiple client requests simultaneously without blocking other operations. This improves

overall application performance and enables efficient utilization of server resources.

In the proposed AI-Powered Sales Forecasting and Business Intelligence Platform, FastAPI serves as the backend application responsible for handling all business logic. It exposes RESTful APIs that allow secure communication between the frontend dashboard and the PostgreSQL database. These APIs perform user authentication, sales management, CRUD operations, audit logging, forecasting requests, and report generation.

The major FastAPI functionalities implemented in the project include:

- User authentication.
- JWT token generation.
- User authorization.
- Restaurant sales management.
- Bill management.
- Historical sales retrieval.
- Forecast generation.
- Audit logging.
- Soft deletion and restoration.
- Secure REST APIs.

The framework also provides automatic request validation through Pydantic models, ensuring that only valid data is accepted by the application. This reduces runtime errors and improves application security.

The automatic Swagger documentation generated by FastAPI significantly simplifies API testing during development and deployment. Developers can verify endpoint functionality directly through the browser without requiring external API testing tools.

Overall, FastAPI provides an ideal backend framework for enterprise-grade applications requiring high performance, security, scalability, and maintainability.

2.12 Streamlit Framework

Streamlit is an open-source Python framework designed for developing interactive web applications and Business Intelligence dashboards with minimal frontend development effort. Unlike traditional web development frameworks that require HTML, CSS, and JavaScript, Streamlit allows developers to build professional dashboards entirely using Python.

Streamlit automatically converts Python scripts into responsive web applications, making it highly suitable for data science, machine learning, and business analytics projects.

The framework provides numerous visualization components including:

- Interactive charts
- KPI cards
- Tables
- Sliders
- Date filters
- Drop-down menus
- Sidebars
- Expandable sections
- File upload widgets
- Download buttons

These components simplify dashboard development while providing an intuitive user experience.

The proposed project uses Streamlit as the primary frontend interface for Business Intelligence visualization. Managers interact with the application through Streamlit dashboards that display business insights generated from PostgreSQL data.

The implemented dashboard includes several analytical modules:

- Overview Dashboard
- Sales Analytics
- Forecast Dashboard
- Model Monitoring
- Reports
- AI Assistant
- Demand Analysis
- Anomaly Detection

Interactive filtering allows users to analyze sales information based on:

- Date ranges
- Categories
- Payment methods
- Meal periods
- Service types
- Weather conditions

The dashboard also provides downloadable PDF and Excel reports that summarize business performance.

The primary advantages of Streamlit include:

- Rapid dashboard development.
- Pure Python implementation.
- Interactive visualization.
- Built-in caching.
- Easy deployment.
- Responsive interface.

- Excellent Plotly integration.

Because of these features, Streamlit has become one of the most widely adopted frameworks for Business Intelligence applications and machine learning dashboards.

2.13 PostgreSQL Database

PostgreSQL is a powerful open-source Relational Database Management System (RDBMS) widely recognized for its reliability, performance, scalability, and compliance with SQL standards. It supports complex queries, transactions, indexing, stored procedures, triggers, and advanced security mechanisms, making it suitable for enterprise-level applications.

Unlike lightweight databases, PostgreSQL ensures data integrity through ACID (Atomicity, Consistency, Isolation, Durability) properties. These characteristics guarantee reliable transaction processing even under concurrent access conditions.

The proposed system uses PostgreSQL as the primary database for storing restaurant operational data.

The database stores several categories of information including:

- User credentials.
- Restaurant bills.
- Bill items.
- Historical sales.
- Audit logs.
- Authentication information.

The major database tables implemented include:

- users
- restaurant_bills
- restaurant_bill_items
- restaurant_sales

- audit_logs

Relationships between these tables maintain data consistency while supporting efficient analytical queries.

The application also imports historical restaurant sales from CSV datasets into PostgreSQL, enabling long-term trend analysis and forecasting. More than two million historical records can be managed efficiently using PostgreSQL's indexing and query optimization capabilities.

PostgreSQL was selected because of its:

- High reliability.
- Open-source licensing.
- Strong security.
- Excellent SQL support.
- Scalability.
- ACID compliance.
- Integration with SQLAlchemy.

These features make PostgreSQL an excellent database platform for Business Intelligence applications.

2.14 SQLAlchemy ORM

SQLAlchemy is a Python Object Relational Mapping (ORM) library that simplifies communication between Python applications and relational databases. Instead of writing raw SQL statements for every database operation, SQLAlchemy enables developers to manipulate database records using Python objects.

SQLAlchemy provides two major functionalities:

- SQL Expression Language.
- Object Relational Mapping (ORM).

The ORM automatically maps Python classes to database tables, significantly reducing development effort while improving code readability and maintainability.

The proposed application uses SQLAlchemy to perform database operations including:

- Data insertion.
- Data retrieval.
- Data updating.
- Data deletion.
- Transaction management.
- Connection pooling.

SQLAlchemy also supports parameterized queries, reducing the risk of SQL Injection attacks and improving application security.

Its integration with PostgreSQL provides high-performance database communication while maintaining portability across multiple database systems.

2.15 RESTful API Architecture

Representational State Transfer (REST) is an architectural style used for designing scalable web services. REST APIs enable communication between different software components through HTTP protocols.

REST defines standard HTTP methods including:

- GET
- POST
- PUT
- DELETE
- PATCH

Each method performs a specific operation on application resources.

In the proposed project, FastAPI exposes several RESTful APIs responsible for:

- User Login
- Sales Creation
- Sales Update
- Sales Retrieval
- Soft Delete
- Record Restoration
- Forecast Generation
- Report Generation
- Dashboard Data Retrieval

REST architecture provides several advantages:

- Platform independence.
- Scalability.
- Stateless communication.
- Easy integration.
- High performance.
- Cloud compatibility.

The frontend dashboard communicates with the backend exclusively through REST APIs, ensuring modularity and maintainability.

2.16 JWT Authentication and Role-Based Access Control

Security is a fundamental requirement for enterprise applications. The proposed system implements JSON Web Token (JWT) authentication to ensure secure communication between clients and backend services.

JWT is a compact, digitally signed token that contains authenticated user information. After successful login, the server generates a token that is attached to every subsequent API request.

The authentication process consists of:

1. User login.
2. Credential verification.
3. JWT generation.
4. Token transmission.
5. Protected API access.
6. Token validation.

The application also implements Role-Based Access Control (RBAC), where each authenticated user is assigned a predefined role.

Supported roles include:

- Administrator
- Manager
- Cashier

Each role possesses different permissions.

For example:

Administrator:

- Full system access.
- User management.
- Sales management.
- Reports.
- Audit logs.

Manager:

- Sales management.
- Dashboard access.
- Forecast viewing.
- Reports.

Cashier:

- Billing operations.
- Sales creation.
- Limited dashboard access.

RBAC enhances application security by ensuring users can perform only authorized operations.

2.17 Cloud Deployment Using Render

Cloud deployment enables software applications to be accessed from anywhere using the internet. Render is a cloud application hosting platform that supports deployment of Python web applications, REST APIs, and databases with minimal configuration.

The proposed system is deployed using Render to demonstrate cloud accessibility and production readiness.

Render provides:

- Continuous deployment.
- GitHub integration.
- Automatic builds.
- HTTPS support.
- Environment variable management.
- Application monitoring.
- Free deployment tier.

The deployment workflow includes:

1. Source code hosted on GitHub.
2. Render connected to GitHub repository.
3. Automatic dependency installation.
4. Application build process.
5. Deployment.

6. Public URL generation.

The project deploys both:

- FastAPI backend.
- Streamlit Business Intelligence dashboard.

This deployment demonstrates the scalability and accessibility of the developed application while supporting future migration to managed cloud databases and AI services.

2.18 Comparative Analysis of Existing Systems

Several restaurant management systems and business intelligence platforms are currently available in the market. While these systems provide essential operational functionalities such as billing, inventory management, and financial reporting, they often lack comprehensive predictive analytics and integrated decision-support capabilities.

A comparative analysis was conducted to evaluate the strengths and limitations of existing restaurant management solutions with respect to the proposed AI-Powered Sales Forecasting and Business Intelligence Platform.

Table 2.1 Comparison of Existing Systems with the Proposed System

Feature	Traditional POS System	Restaurant ERP	BI Dashboard	Proposed System
Billing Management	✓	✓	✗	✓
Customer Management	Limited	✓	✗	✓
Historical Sales Analysis	Basic	Moderate	✓	✓
Interactive Dashboard	✗	Limited	✓	✓

Feature	Traditional POS System	Restaurant ERP	BI Dashboard	Proposed System
Sales Forecasting	✗	Limited	Limited	✓
REST API Support	✗	Limited	✗	✓
JWT Authentication	✗	Limited	✗	✓
Role-Based Access	Limited	✓	✗	✓
Audit Logging	✗	Limited	✗	✓
Soft Delete & Restore	✗	✗	✗	✓
Cloud Deployment	Limited	✓	✓	✓
AI Integration Ready	✗	✗	Limited	✓

The comparison indicates that existing systems primarily emphasize transaction management and historical reporting. Very few platforms integrate business intelligence, predictive forecasting, secure authentication, and cloud deployment within a unified architecture.

The proposed system distinguishes itself by combining operational management with advanced analytical capabilities. Features such as Prophet-based forecasting, interactive Streamlit dashboards, audit logging, JWT authentication, and RESTful APIs provide a comprehensive solution that addresses both operational and strategic business requirements.

2.19 Research Gap

The literature survey identified several limitations in existing restaurant management systems and analytical platforms.

Lack of Predictive Forecasting

Many restaurant management systems focus on recording historical transactions but do not provide mechanisms for forecasting future sales. Managers must rely on intuition or manual calculations when planning inventory and staffing.

Limited Business Intelligence

Although several Business Intelligence platforms provide interactive dashboards, they often require external data processing tools and lack direct integration with operational restaurant management systems.

Inadequate Integration

Existing systems frequently separate operational modules from analytical modules. This fragmented architecture increases complexity and reduces the efficiency of managerial decision-making.

Security Limitations

Numerous small-scale restaurant management applications provide minimal authentication mechanisms and lack Role-Based Access Control (RBAC), making them unsuitable for enterprise environments.

Absence of Audit Trails

Many systems do not maintain comprehensive audit logs to monitor user activities, making accountability and security management difficult.

No Soft Delete Mechanism

Traditional systems permanently remove records during deletion operations, increasing the risk of accidental data loss. Soft deletion provides a safer approach by preserving deleted records for future recovery.

Limited Scalability

Several legacy restaurant management systems rely on desktop-based architectures that cannot easily support cloud deployment, mobile accessibility, or distributed operations.

AI Integration Challenges

Artificial Intelligence has become increasingly important for business analytics; however, most existing restaurant management applications do not provide extensible architectures capable of integrating AI-powered recommendation systems or conversational assistants.

Research Gap Addressed by the Proposed System

The proposed AI-Powered Sales Forecasting and Business Intelligence Platform addresses these research gaps by integrating:

- Secure JWT Authentication.
- Role-Based Access Control.
- FastAPI RESTful Backend.
- PostgreSQL Database.
- Streamlit Business Intelligence Dashboard.
- Interactive KPI Visualization.
- Historical Sales Analysis.
- Prophet-based Time-Series Forecasting.
- Audit Logging.
- Soft Delete and Record Restoration.
- Modular Software Architecture.
- Cloud Deployment using Render.
- Future AI Assistant Integration.

By combining these technologies into a single platform, the developed application provides a modern decision-support system capable of improving operational efficiency and enabling data-driven restaurant management.

2.20 Chapter Summary

This chapter presented a comprehensive review of existing research, technologies, and methodologies related to restaurant management systems, business intelligence platforms, predictive analytics, sales forecasting, cloud computing, and modern software development frameworks.

The study demonstrated that traditional restaurant management systems primarily focus on operational activities such as billing and transaction recording while providing limited analytical capabilities. Business Intelligence platforms enhance data visualization but often lack direct integration with operational databases. Time-series forecasting techniques, particularly Facebook Prophet, have proven effective for predicting business performance due to their ability to model trends, seasonality, and holiday effects.

The chapter also discussed the importance of FastAPI, Streamlit, PostgreSQL, SQLAlchemy, RESTful APIs, JWT authentication, Role-Based Access Control, and cloud deployment technologies in developing scalable enterprise applications.

A comparative analysis highlighted the limitations of existing systems and established the necessity of an integrated Business Intelligence platform capable of combining operational management with predictive analytics. The identified research gaps justified the development of the proposed AI-Powered Sales Forecasting and Business Intelligence Platform.

The knowledge gained through this literature survey provides the theoretical foundation for the subsequent chapters, which describe the analysis, design, implementation, testing, and deployment of the developed system.

CHAPTER 3

TECHNOLOGIES USED

3.1 Introduction

The successful development of any modern software application depends on selecting an appropriate technology stack capable of meeting both functional and non-functional requirements. The AI-Powered Sales Forecasting and Business Intelligence Platform integrates multiple open-source technologies to develop a scalable, secure, high-performance, and user-friendly application for restaurant management.

The proposed system combines modern backend development frameworks, interactive dashboard technologies, enterprise-level database systems, machine learning algorithms, cloud deployment platforms, and secure authentication mechanisms into a unified software architecture. Each technology contributes to a specific component of the application while collectively ensuring modularity, maintainability, scalability, and reliability.

The project follows a layered architecture consisting of:

- Presentation Layer
- Business Logic Layer
- Data Access Layer
- Database Layer
- Forecasting Layer
- Deployment Layer

Each layer utilizes specialized technologies optimized for its specific functionality.

The major technologies implemented in this project include:

- Python
- FastAPI

- Streamlit
- PostgreSQL
- SQLAlchemy
- Pydantic
- JWT Authentication
- bcrypt Password Hashing
- Prophet Forecasting
- Pandas
- NumPy
- Plotly
- Matplotlib
- Git & GitHub
- Render Cloud Platform
- CSV Data Processing
- REST API Architecture
- Environment Variables

This chapter discusses each technology in detail.

3.2 Python Programming Language

Introduction

Python is a high-level, interpreted, object-oriented programming language developed by Guido van Rossum in 1991. It has become one of the most popular programming languages due to its simple syntax, readability, extensive library ecosystem, and strong community support.

Python is widely used in:

- Web Development
- Machine Learning

- Artificial Intelligence
- Data Science
- Automation
- Scientific Computing
- Cloud Computing
- Cybersecurity
- Business Intelligence

Its simplicity enables developers to build complex applications with significantly less code compared to traditional programming languages.

Features of Python

Python provides numerous features that make it suitable for enterprise application development.

- Easy-to-read syntax.
 - Platform independence.
 - Object-Oriented Programming.
 - Large Standard Library.
 - Dynamic typing.
 - Automatic memory management.
 - Cross-platform compatibility.
 - Extensive third-party packages.
 - Strong community support.
-

Advantages

- Faster software development.
- Excellent machine learning support.

- Large open-source ecosystem.
 - Easy API development.
 - Powerful database integration.
 - Excellent visualization libraries.
 - Cloud deployment compatibility.
-

Disadvantages

- Slower execution than compiled languages.
 - Higher memory consumption.
 - Dynamic typing may introduce runtime errors.
 - Less suitable for low-level system programming.
-

Why Python was selected

Python was selected because the project combines:

- Business Intelligence
- Dashboard Development
- Machine Learning
- Database Management
- API Development

Python supports all these technologies within a single programming environment.

Python in the Proposed Project

Python serves as the primary programming language for developing:

- FastAPI backend.
- Streamlit frontend.

- Prophet forecasting.
- Database communication.
- Authentication.
- Report generation.
- Dashboard visualization.
- Historical data processing.

The complete application has been developed entirely using Python.

3.3 FastAPI Framework

Introduction

FastAPI is a modern Python framework used for developing high-performance RESTful APIs. Developed by Sebastián Ramírez in 2018, FastAPI has become one of the fastest-growing backend frameworks due to its speed, simplicity, and automatic API documentation.

FastAPI is built upon:

- Starlette
- Pydantic

These technologies provide asynchronous programming capabilities and automatic request validation.

Architecture of FastAPI

FastAPI follows a layered architecture consisting of:

Client

↓

HTTP Request

↓

FastAPI Router

↓

Business Logic

↓

Database Layer

↓

HTTP Response

This architecture separates application logic from database operations, improving maintainability.

Features

- High performance.
- Automatic Swagger documentation.
- ReDoc API documentation.
- Asynchronous request handling.
- Dependency Injection.
- Data validation.
- Type hint support.
- Security integration.

Advantages

- Extremely fast.
- Automatic documentation.
- Easy REST API development.
- Excellent scalability.
- Modern Python syntax.
- High security.

Why FastAPI was selected

The project requires:

- Secure login.
- CRUD operations.
- Dashboard APIs.
- Forecast APIs.
- Authentication.

FastAPI efficiently supports all these requirements.

FastAPI in the Proposed System

FastAPI handles:

- User Login.
- JWT Authentication.
- Bill Creation.
- Bill Update.
- Soft Delete.
- Restore.
- Sales Retrieval.
- Historical Data.
- Forecast APIs.
- Audit Logs.

Every dashboard request is processed through FastAPI APIs.

3.4 Streamlit Framework

Introduction

Streamlit is an open-source Python framework used to develop interactive web dashboards and Business Intelligence applications without requiring HTML, CSS, or JavaScript.

It transforms Python scripts into fully interactive web applications.

Features

- Sidebar.
 - Interactive filters.
 - KPI cards.
 - Charts.
 - Download buttons.
 - File upload.
 - Session state.
 - Multipage applications.
-

Advantages

- Rapid dashboard development.
 - Pure Python.
 - Interactive visualization.
 - Responsive layout.
 - Easy deployment.
-

Streamlit in Proposed Project

The dashboard consists of:

Overview



Analytics



Forecast



Reports



AI Assistant



Model Monitoring



Demand Analysis



Anomaly Detection

Managers interact with Streamlit to analyze restaurant performance.

3.5 PostgreSQL Database

Introduction

PostgreSQL is an enterprise-level Relational Database Management System (RDBMS) that supports SQL standards, ACID transactions, indexing, concurrency control, and advanced security.

It is one of the world's most reliable open-source databases.

Features

- ACID Compliance.
- Indexing.
- Transactions.

- Constraints.
 - Triggers.
 - Views.
 - Stored Procedures.
 - Foreign Keys.
-

Advantages

- High performance.
 - Excellent reliability.
 - Large dataset support.
 - Secure.
 - Free and open-source.
-

PostgreSQL in Proposed Project

The application stores:

Users

↓

Bills

↓

Bill Items

↓

Restaurant Sales

↓

Audit Logs

Historical data containing millions of records is stored within PostgreSQL.

3.6 SQLAlchemy ORM

Introduction

SQLAlchemy is a Python Object Relational Mapper (ORM) that connects Python applications with relational databases.

Instead of writing SQL for every query, developers use Python functions.

Advantages

- Database independence.
 - SQL Injection protection.
 - Connection pooling.
 - Clean architecture.
 - Easy maintenance.
-

SQLAlchemy in Proposed Project

Used for:

- Insert Bills.
 - Update Bills.
 - Delete Bills.
 - Restore Bills.
 - Retrieve Sales.
 - Forecast Data.
 - Authentication.
-

3.7 Pydantic

Introduction

Pydantic is a data validation library used by FastAPI.

It validates incoming API requests automatically.

Advantages

- Automatic validation.
 - Type checking.
 - Error handling.
 - JSON serialization.
-

Project Usage

Used for validating:

- Login requests.
- Bill requests.
- Forecast requests.
- Dashboard requests.

This prevents invalid data from entering the database.

3.8 JWT Authentication

Introduction

JSON Web Token (JWT) is a secure authentication mechanism that enables stateless user authentication.

Working

User Login

↓

Verify Password

↓

Generate JWT

↓

Return Token

↓

Protected APIs

↓

Verify Token

Advantages

- Stateless.
 - Secure.
 - Fast.
 - Scalable.
-

Project Usage

JWT secures:

- Dashboard.
 - CRUD.
 - Reports.
 - Forecast.
 - User Management.
-

3.9 bcrypt Password Hashing

Passwords should never be stored in plain text.

bcrypt converts passwords into irreversible encrypted hashes.

Advantages:

- Strong encryption.
- Salt generation.
- Brute-force resistance.
- Industry standard.

Project usage:

Every user password stored inside PostgreSQL is hashed using bcrypt.

3.10 Pandas

Pandas is the primary data analysis library.

Functions:

- CSV Import.
- Data Cleaning.
- Aggregation.
- Feature Engineering.
- Time-Series Processing.

Project usage:

Historical restaurant sales are processed using Pandas before forecasting.

3.11 NumPy

NumPy provides efficient numerical computation.

Used for:

- Mathematical calculations.
 - Forecast preprocessing.
 - Statistical analysis.
-

3.12 Plotly

Plotly provides interactive graphs.

Used in dashboard:

- Revenue Trends.
 - Category Analysis.
 - Payment Analysis.
 - Forecast Graphs.
 - KPI Charts.
-

3.13 Matplotlib

Used for:

- Forecast visualization.
 - Static charts.
 - Report generation.
-

3.14 Prophet

Prophet is Meta's forecasting library.

Project usage:

Historical Sales

↓

Train Model

↓

Generate Future Dates

↓

Predict Revenue

↓

Display Forecast

Advantages:

- Seasonality detection.
 - Holiday effects.
 - Confidence intervals.
-

3.15 Git & GitHub

Git provides version control.

GitHub stores source code online.

Used for:

- Version history.
 - Collaboration.
 - Deployment.
-

3.16 Render Cloud Platform

Render hosts:

- FastAPI Backend.
- Streamlit Dashboard.

Advantages:

- Free deployment.
 - HTTPS.
 - Continuous deployment.
 - GitHub integration.
-

3.17 Environment Variables

Sensitive information is stored securely using environment variables.

Examples:

- DATABASE_URL
- SECRET_KEY

This prevents credentials from being hardcoded into source code.

3.18 Chapter Summary

This chapter discussed all the major technologies used in the development of the AI-Powered Sales Forecasting and Business Intelligence Platform. Python served as the core programming language, while FastAPI provided backend services and Streamlit enabled interactive dashboard development. PostgreSQL managed relational data, SQLAlchemy simplified database operations, and Pydantic ensured request validation. Security was implemented through JWT authentication and bcrypt password hashing. Prophet supported predictive sales forecasting, while Pandas, NumPy, Plotly, and Matplotlib facilitated data processing and visualization. GitHub and Render enabled version control and cloud deployment, resulting in a modern, scalable, and secure software solution suitable for restaurant business intelligence and decision support.

CHAPTER 4

SYSTEM ANALYSIS

4.1 Introduction

System analysis is one of the most critical phases of the Software Development Life Cycle (SDLC). It involves understanding the existing business process, identifying its limitations, gathering user requirements, analyzing business problems, and designing an improved solution capable of addressing current organizational challenges.

The primary objective of system analysis is to understand how an organization currently performs its operations and determine how technology can improve efficiency, productivity, security, and decision-making.

In this project, a detailed analysis was conducted on restaurant sales management processes to identify operational inefficiencies and opportunities for business intelligence integration. The analysis revealed that while most restaurants successfully manage billing operations using conventional Point-of-Sale (POS) systems, they often lack predictive analytics, interactive dashboards, secure user management, and intelligent forecasting capabilities.

The proposed AI-Powered Sales Forecasting and Business Intelligence Platform was designed to overcome these limitations by integrating sales management, forecasting, authentication, business intelligence, audit logging, and cloud deployment into a unified software solution.

4.2 Existing System Analysis

The existing restaurant management process primarily focuses on recording customer transactions and generating daily financial reports. Most small and medium-sized restaurants rely on conventional billing software or spreadsheet-based record keeping to maintain operational information.

Although these systems efficiently perform billing operations, they provide only descriptive reports that summarize historical sales information. Managers must

manually analyze these reports to understand business performance and estimate future demand.

Existing systems generally perform the following operations:

- Customer billing
- Menu management
- Payment recording
- Receipt generation
- Daily sales reports
- Basic financial summaries

However, these systems do not provide intelligent analytical capabilities that assist management in strategic planning.

Existing System Workflow

Customer Order



POS Billing



Database Storage



Static Reports



Manual Decision Making

The existing workflow demonstrates that decision-making relies primarily on manual interpretation of historical reports rather than automated analytical insights.

4.3 Limitations of Existing System

A detailed analysis identified several limitations associated with traditional restaurant management systems.

1. Lack of Forecasting

Existing systems cannot predict future sales or customer demand.

Restaurant managers must estimate future requirements manually.

2. Manual Report Analysis

Business reports are generated in static formats.

Managers spend considerable time analyzing reports before making decisions.

3. Limited Business Intelligence

Traditional systems lack interactive dashboards capable of visualizing business performance dynamically.

4. Weak Security

Several systems provide only username-password authentication without Role-Based Access Control (RBAC).

5. No Audit Logging

User activities are generally not monitored.

Unauthorized modifications cannot easily be identified.

6. Permanent Record Deletion

Deleted records are permanently removed.

Accidental deletions cannot be recovered.

7. Poor Scalability

Desktop-based architectures cannot easily support cloud deployment or future AI integration.

8. Limited Decision Support

Managers receive historical information but no predictive recommendations.

4.4 Proposed System Analysis

The proposed AI-Powered Sales Forecasting and Business Intelligence Platform addresses the limitations identified during system analysis by integrating modern software engineering practices with Business Intelligence technologies.

The proposed system consists of several independent yet interconnected modules.

Users



JWT Authentication



FastAPI Backend



Business Logic



PostgreSQL Database



Business Intelligence Dashboard



Forecasting Module



Decision Support

Each component performs a specialized function while communicating through secure REST APIs.

4.5 Advantages of Proposed System

The developed platform provides numerous advantages over traditional restaurant management systems.

Operational Advantages

- Secure billing management.
- Centralized database.
- Automated calculations.

- Better transaction management.
-

Analytical Advantages

- Interactive dashboards.
 - Historical trend analysis.
 - KPI visualization.
 - Revenue analytics.
-

Forecasting Advantages

- Future sales prediction.
 - Seasonal analysis.
 - Revenue estimation.
 - Business planning support.
-

Security Advantages

- JWT Authentication.
 - Password encryption.
 - Role-Based Access Control.
 - Audit logging.
-

Administrative Advantages

- Soft Delete.
 - Record Restoration.
 - User Management.
 - Cloud Deployment.
-

4.6 Functional Requirements

Functional requirements describe the services that the application must provide.

The major functional requirements are summarized below.

Authentication Module

- User Login
 - JWT Token Generation
 - Token Verification
 - Role-Based Authorization
-

Sales Management Module

- Create Bills
 - Update Bills
 - Delete Bills
 - Restore Bills
 - Retrieve Bills
-

Dashboard Module

- KPI Cards
 - Revenue Charts
 - Category Analysis
 - Payment Analysis
 - Historical Trends
-

Forecast Module

- Train Prophet Model

- Generate Future Forecast
 - Display Forecast Graphs
 - Forecast Download
-

Reports Module

- PDF Export
 - Excel Export
 - Dashboard Reports
-

Audit Module

- Store User Activities
 - Track CRUD Operations
 - Maintain Audit History
-

4.7 Non-Functional Requirements

The developed system must satisfy several quality requirements.

Performance

The application should process API requests efficiently and generate dashboard visualizations with minimal delay.

Reliability

The system should maintain data consistency and provide uninterrupted service during normal operation.

Scalability

The architecture should support future expansion including AI assistants, cloud databases, and multi-branch management.

Security

User authentication, encrypted passwords, secure APIs, and Role-Based Access Control should protect sensitive information.

Maintainability

The application should follow a modular architecture that simplifies debugging, updates, and future enhancements.

Usability

The dashboard should provide an intuitive interface enabling restaurant managers to analyze business information without requiring technical expertise.

4.8 Feasibility Study

A feasibility study was conducted before system development.

Technical Feasibility

The selected technologies including Python, FastAPI, PostgreSQL, Streamlit, Prophet, and Render are open-source, well-documented, and widely adopted within industry.

Therefore, the project is technically feasible.

Economic Feasibility

The software utilizes open-source technologies.

No commercial software licenses are required.

Deployment on Render Free Tier further minimizes operational costs.

Hence, the project is economically feasible.

Operational Feasibility

Restaurant managers require accurate forecasting and business analytics.

The developed application satisfies operational requirements by providing intuitive dashboards and automated forecasting.

Schedule Feasibility

The project was successfully completed within the internship duration using Agile development methodology.

4.9 Software Requirement Specification (SRS)

Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or above
 - RAM: Minimum 8 GB
 - Storage: 20 GB Free Space
 - Internet Connection
 - Modern Web Browser
-

Software Requirements

- Windows 10/11 or Linux
- Python 3.13
- PostgreSQL
- VS Code
- Git

- GitHub
 - FastAPI
 - Streamlit
 - SQLAlchemy
 - Prophet
 - Render
-

4.10 User Roles

The application supports three user roles.

Administrator

- Full access
 - User management
 - Dashboard
 - CRUD
 - Reports
 - Audit Logs
-

Manager

- Dashboard
 - Forecast
 - Reports
 - Sales Management
-

Cashier

- Billing
- Customer Orders

- Limited Dashboard
-

4.11 Use Case Analysis

The following use cases are supported.

Administrator

- Login
 - Manage Users
 - Create Bills
 - Update Bills
 - Delete Bills
 - Restore Bills
 - View Dashboard
 - Forecast Sales
 - Export Reports
 - View Audit Logs
-

Manager

- Login
 - Dashboard
 - Reports
 - Forecast
 - Sales
-

Cashier

- Login
- Billing

- Customer Management
-

4.12 Chapter Summary

This chapter analyzed the existing restaurant management process and identified its operational limitations. The proposed AI-Powered Sales Forecasting and Business Intelligence Platform was designed to overcome these shortcomings by integrating secure authentication, business intelligence, predictive forecasting, centralized data management, and cloud-ready architecture. Functional requirements, non-functional requirements, feasibility studies, and software requirements were presented to justify the proposed solution. The analysis confirms that the developed system is technically feasible, economically viable, operationally effective, and scalable for future enhancements.

CHAPTER 5

SYSTEM DESIGN

5.1 Introduction

System design is a critical phase of the Software Development Life Cycle (SDLC), where the requirements identified during system analysis are transformed into a structured technical solution. It provides a blueprint for implementing the application by defining the software architecture, database structure, module interactions, communication mechanisms, and workflow processes.

The AI-Powered Sales Forecasting and Business Intelligence Platform follows a modular, layered architecture to ensure maintainability, scalability, and security. Each module is designed to perform a specific business function while communicating efficiently with other components through RESTful APIs.

The architecture consists of the following layers:

- Presentation Layer
- API Layer
- Business Logic Layer
- Data Access Layer
- Database Layer
- Forecasting Layer

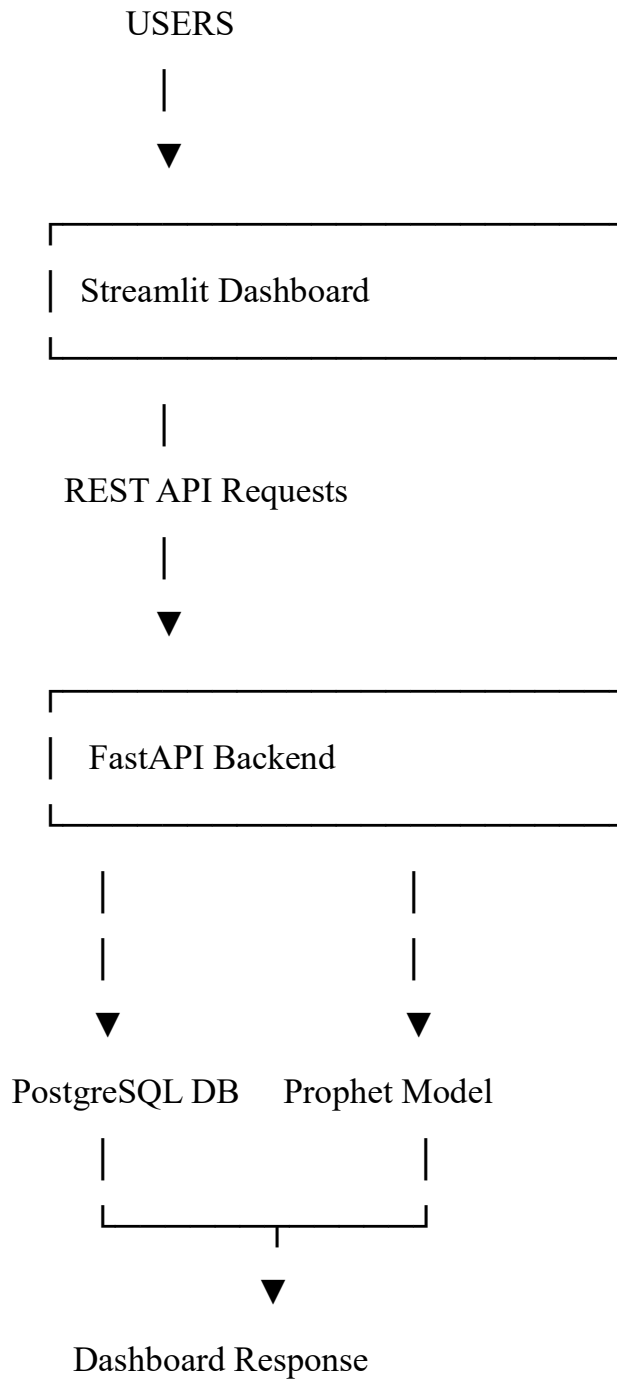
This separation of responsibilities simplifies maintenance and enables future expansion without affecting existing functionalities.

5.2 Overall System Architecture

The overall architecture of the proposed system follows a client-server model. The Streamlit dashboard acts as the client interface, while FastAPI serves as the backend application responsible for business logic. PostgreSQL stores

operational data, and the Prophet forecasting engine generates future sales predictions.

Figure 5.1 Overall System Architecture



Explanation

The user interacts with the Streamlit dashboard through a web browser. User actions such as login, viewing analytics, generating forecasts, or managing bills are sent as REST API requests to the FastAPI backend.

FastAPI processes these requests, communicates with PostgreSQL to retrieve or update data, and invokes the Prophet forecasting model whenever predictions are required. The processed information is then returned to the Streamlit dashboard for visualization.

This architecture ensures clear separation between presentation, business logic, and data management.

5.3 Layered Architecture

The system follows a six-layer architecture.

Presentation Layer

Responsible for displaying dashboards, charts, filters, reports, and forms.

Technology Used:

- Streamlit
- Plotly

Functions:

- User Login
- Dashboard
- Reports
- Forecast
- Analytics

API Layer

Acts as the communication bridge between frontend and backend.

Technology Used:

- FastAPI

Responsibilities:

- Handle HTTP requests
 - Authenticate users
 - Validate requests
 - Route requests
-

Business Logic Layer

Contains all business rules.

Responsibilities:

- Bill processing
 - Revenue calculation
 - Forecast execution
 - Audit logging
 - CRUD operations
-

Data Access Layer

Responsible for database communication.

Technology Used:

- SQLAlchemy

Responsibilities:

- Insert
 - Update
 - Delete
 - Query
-

Database Layer

Stores all application data.

Technology Used:

- PostgreSQL

Tables:

- users
 - restaurant_bills
 - restaurant_bill_items
 - restaurant_sales
 - audit_logs
-

Forecast Layer

Responsible for prediction.

Technology Used:

- Prophet

Functions:

- Train model
 - Forecast revenue
 - Evaluate predictions
-

5.4 Module Design

The developed application consists of several independent modules.

Authentication Module

Functions

- Login
- Password Verification

- JWT Token Generation
 - Authorization
 - Role Validation
-

Sales Management Module

Functions

- Create Bill
 - Update Bill
 - Delete Bill
 - Restore Bill
 - Sales Retrieval
-

Dashboard Module

Functions

- KPI Cards
 - Revenue Analysis
 - Category Analysis
 - Payment Analysis
 - Meal Period Analysis
 - Weather Analysis
-

Forecast Module

Functions

- Data Preprocessing
- Prophet Training
- Revenue Prediction

- Forecast Visualization

Reports Module

Functions

- PDF Reports
- Excel Reports
- Download Forecast

Audit Module

Functions

- User Activity Logging
- CRUD Tracking
- Timestamp Recording

5.5 Authentication Workflow

Figure 5.2 Authentication Flow

User

|



Enter Username & Password

|



FastAPI Login API



Verify Password (bcrypt)



Generate JWT Token



Return Token



Protected API Access

Explanation

Users enter their login credentials through the dashboard. FastAPI verifies the username and password stored in PostgreSQL using bcrypt hashing. If authentication succeeds, a JWT token is generated and returned to the client. This token is attached to subsequent API requests, allowing secure access to protected resources.

5.6 CRUD Workflow

Figure 5.3 CRUD Process

Dashboard

|



REST API

|



Business Logic

|



Database

|



Success Response

|



Dashboard Update

Operations Supported

- Create
- Read
- Update
- Soft Delete
- Restore

5.7 Forecast Workflow

Figure 5.4 Forecast Workflow

Historical Sales

|



Data Cleaning



Feature Engineering



Prophet Training



Future Dates



Forecast

|



Visualization

Explanation

Historical restaurant sales stored within PostgreSQL are preprocessed using Pandas. The cleaned dataset is passed to Prophet, which trains a forecasting model and predicts future revenue. Forecast results are displayed as interactive graphs on the dashboard.

5.8 Dashboard Workflow

User

|



Dashboard

|



Apply Filters

|



API Request

|



Database

|



Analytics

|



Charts

Interactive filters allow managers to explore historical business data efficiently.

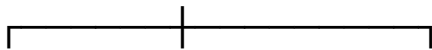
5.9 Component Diagram

Figure 5.5 Component Diagram

Streamlit Dashboard



FastAPI Backend



Auth

Sales

Forecast



PostgreSQL Prophet

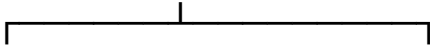
5.10 Deployment Diagram

Figure 5.6 Deployment Diagram

GitHub Repository



Render Deployment



FastAPI

Streamlit



PostgreSQL

(Local during development /
Cloud-ready architecture)

Explanation

The source code is maintained using GitHub. Both the FastAPI backend and Streamlit dashboard are deployed independently on Render. During development, PostgreSQL runs locally; the architecture supports migration to a managed cloud PostgreSQL service such as Supabase or Render PostgreSQL.

5.11 Database Design Overview

The database follows normalization principles to eliminate redundancy and maintain data consistency.

Primary Tables

Table	Purpose
users	Stores user credentials and roles
restaurant_bills	Stores bill header information
restaurant_bill_items	Stores item-level bill details
restaurant_sales	Stores analytical sales records
audit_logs	Stores user activity logs

The database uses primary keys, foreign keys, and constraints to ensure referential integrity and reliable transaction processing.

5.12 API Design

The backend exposes RESTful APIs for communication between the frontend and server.

Authentication APIs

- POST /login

Sales APIs

- POST /sales
- GET /sales
- GET /sales/{bill_no}

- PUT /sales/{bill_no}
- DELETE /sales/{bill_no}
- PUT /sales/restore/{bill_no}

Dashboard APIs

- GET /dashboard
- GET /forecast

These APIs provide a standardized interface for all application modules.

5.13 Security Design

The system incorporates multiple security mechanisms:

- JWT-based authentication
- bcrypt password hashing
- Role-Based Access Control (RBAC)
- Protected REST endpoints
- Audit logging
- Environment variable management for secrets

These measures help protect sensitive data and restrict unauthorized access.

5.14 Chapter Summary

This chapter described the overall design of the AI-Powered Sales Forecasting and Business Intelligence Platform. It presented the layered architecture, system modules, authentication workflow, CRUD workflow, forecasting process, dashboard interaction, deployment architecture, database overview, API structure, and security mechanisms. The modular design improves maintainability, scalability, and reliability while providing a solid foundation for future enhancements such as cloud databases, AI assistants, inventory management, and multi-branch support.

CHAPTER 6

DATABASE DESIGN

6.1 Introduction

A database is the backbone of any information system, providing a structured and secure mechanism for storing, organizing, retrieving, and managing application data. In modern enterprise applications, an efficient database design is essential for maintaining data consistency, minimizing redundancy, ensuring data integrity, and supporting high-performance query execution.

The AI-Powered Sales Forecasting and Business Intelligence Platform uses **PostgreSQL**, an enterprise-grade Relational Database Management System (RDBMS), to store restaurant operational data, analytical information, authentication details, and audit logs. The database has been designed following normalization principles to ensure efficient storage, eliminate unnecessary duplication, and simplify future scalability.

The database supports both operational transaction processing and analytical reporting. Operational tables manage restaurant billing and user authentication, while analytical tables maintain transformed sales records used for Business Intelligence dashboards and sales forecasting.

The database architecture consists of five primary tables:

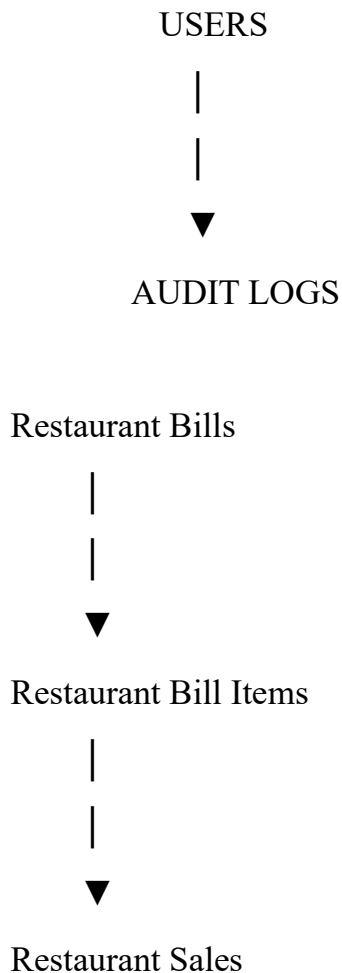
- Users
- Restaurant Bills
- Restaurant Bill Items
- Restaurant Sales
- Audit Logs

Relationships between these tables enable efficient retrieval of information while maintaining referential integrity through primary and foreign key constraints.

6.2 Database Architecture

The proposed database architecture follows a relational model where multiple tables are connected through well-defined relationships.

Figure 6.1 Database Architecture



Explanation

The **Users** table stores authentication credentials and user roles. Every significant user action is recorded in the **Audit Logs** table.

Restaurant transactions begin with a bill stored in the **Restaurant Bills** table. Each bill contains multiple menu items, which are stored in the **Restaurant Bill Items** table. These transactional records are transformed into analytical entries within the **Restaurant Sales** table, enabling dashboard visualization and forecasting.

6.3 Database Normalization

Database normalization is the process of organizing data to reduce redundancy and improve consistency.

The proposed database follows the principles of normalization up to the **Third Normal Form (3NF)**.

First Normal Form (1NF)

- Each table contains atomic values.
- No repeating groups are present.
- Every record has a unique primary key.

Second Normal Form (2NF)

- All non-key attributes depend entirely on the primary key.
- Partial dependencies have been removed.

Third Normal Form (3NF)

- Transitive dependencies are eliminated.
- Each attribute depends only on the primary key.

Normalization improves:

- Storage efficiency.
 - Data consistency.
 - Update performance.
 - Data integrity.
 - Query optimization.
-

6.4 Users Table

The **Users** table stores authentication information for every authorized user of the system.

It supports Role-Based Access Control (RBAC), allowing different permission levels for administrators, managers, and cashiers.

Purpose

- User authentication.
- Password storage.
- Role management.
- Account status.

Primary Key

user_id

Important Fields

Field	Data Type	Description
user_id	BIGINT	Unique user identifier
username	VARCHAR	Login username
password	VARCHAR	bcrypt hashed password
role	VARCHAR	User role
created_at	TIMESTAMP	Account creation time

Advantages

- Secure authentication.
- Role management.
- Centralized user administration.

6.5 Restaurant Bills Table

The **Restaurant Bills** table stores high-level billing information generated during customer transactions.

Each record represents one completed customer bill.

Purpose

Store:

- Bill number.
- Customer information.
- Payment details.
- Revenue.
- Taxes.
- Discounts.
- Service charges.

Primary Key

bill_id

Important Fields

Field	Description
bill_no	Bill Number
bill_date	Transaction Date
customer_name	Customer Name
service_type	Dine-In / Takeaway
payment_method	Cash / Card / UPI
gross_sales	Gross Revenue
discount_amount	Discount
tax_amount	GST
total_billed	Total Amount
net_revenue	Final Revenue

Additional Fields

Soft Delete Implementation:

- is_deleted
- deleted_at
- deleted_by

These fields allow deleted bills to be restored instead of permanently removed.

6.6 Restaurant Bill Items Table

Each restaurant bill may contain multiple menu items.

This table stores item-level transaction information.

Purpose

Maintain:

- Menu Items.
- Categories.
- Quantities.
- Unit Prices.
- Discounts.

Relationship

One Bill

↓

Many Bill Items

Important Fields

Field	Description
bill_item_id	Primary Key
bill_id	Foreign Key
item_name	Menu Item
category	Food Category

Field	Description
menu_group	Menu Group
quantity	Ordered Quantity
item_rate	Unit Price
discount_amount	Discount

6.7 Restaurant Sales Table

The Restaurant Sales table is the analytical dataset used for Business Intelligence and forecasting.

Unlike the Restaurant Bills table, this table stores transformed transactional information optimized for reporting and machine learning.

Purpose

Support:

- Dashboard.
- Analytics.
- Forecast.
- Charts.

Major Fields

Field	Description
Date	Transaction Date
BillNo	Bill Number
Category	Food Category
Item_Name	Product Name
Quantity	Quantity Sold

Field	Description
Unit_Price	Product Price
Gross_Sales	Gross Revenue
Discount_Amt	Discount
Net_Sales	Revenue After Discount
Tax_Amt	Tax
Total_Billed	Final Bill
Net_Revenue	Revenue
Payment_Method	Payment Type
Meal_Period	Breakfast/Lunch/Dinner
Weather	Weather Condition
Is_Holiday	Holiday Flag
Is_Weekend	Weekend Flag

This table forms the primary input for the Prophet forecasting model.

6.8 Audit Logs Table

The Audit Logs table records all critical user activities performed within the application.

Purpose

Maintain:

- User accountability.
- Security.
- System monitoring.

Logged Activities

- Login.
- Bill Creation.
- Bill Update.
- Soft Delete.
- Restore.
- Report Generation.

Fields

Field	Description
audit_id	Primary Key
username	User
role	User Role
action	CRUD Operation
bill_no	Related Bill
description	Activity Description
timestamp	Activity Time

Audit logs improve application transparency and support future compliance requirements.

6.9 Entity Relationship (ER) Diagram

Figure 6.2 ER Diagram

Users

|

|



Audit Logs

Restaurant Bills



Restaurant Bill Items



Restaurant Sales

Relationship Summary

Parent	Child	Relationship
Restaurant Bills	Bill Items	One-to-Many
Users	Audit Logs	One-to-Many

6.10 Database Constraints

The proposed database implements several constraints.

Primary Keys

Ensure uniqueness.

Examples:

- user_id
 - bill_id
 - sale_id
 - audit_id
-

Foreign Keys

Maintain relationships.

Example:

bill_items.bill_id

↓

restaurant_bills.bill_id

NOT NULL Constraints

Critical fields cannot remain empty.

Example:

- username
 - bill_no
 - payment_method
 - total_billed
-

Default Values

Several fields automatically receive values.

Example:

created_at

↓

CURRENT_TIMESTAMP

6.11 Indexing Strategy

Indexes improve query performance.

Indexes are created on:

- bill_no

- bill_date
- payment_method
- category
- Date

Benefits:

- Faster filtering.
- Faster searching.
- Faster dashboard loading.
- Faster forecasting.

6.12 Data Flow Inside Database

Restaurant Billing

|



Restaurant Bills

|



Restaurant Bill Items

|



Restaurant Sales



Dashboard



Forecast

The transformed sales data enables analytical queries without affecting transactional performance.

6.13 Security Considerations

The database incorporates multiple security measures.

- Password hashing using bcrypt.
- JWT Authentication.
- Role-Based Access Control.
- Audit Logging.
- Soft Delete.

- Secure database transactions.
 - SQLAlchemy ORM to reduce SQL injection risks.
-

6.14 Advantages of Database Design

The proposed database offers:

- High scalability.
 - Data integrity.
 - Reduced redundancy.
 - Efficient analytical queries.
 - Secure authentication.
 - Reliable transaction management.
 - Business Intelligence support.
 - Forecast-ready datasets.
 - Easy maintenance.
 - Cloud migration readiness.
-

6.15 Chapter Summary

This chapter presented the database design of the AI-Powered Sales Forecasting and Business Intelligence Platform. The PostgreSQL database was designed using relational principles and normalized to the Third Normal Form (3NF) to ensure consistency and minimize redundancy. Five primary tables—Users, Restaurant Bills, Restaurant Bill Items, Restaurant Sales, and Audit Logs—were described along with their purposes, key fields, relationships, and constraints. The chapter also discussed indexing, security measures, and data flow, demonstrating how the database supports both transactional processing and analytical operations. This robust database foundation enables efficient business intelligence, forecasting, secure authentication, and future scalability.

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 Introduction

System implementation is the phase in the Software Development Life Cycle (SDLC) where the designed architecture is transformed into a fully functional software application. During this phase, all software modules are developed, integrated, tested, and deployed to create a complete working system.

The AI-Powered Sales Forecasting and Business Intelligence Platform was implemented using modern Python-based technologies, including FastAPI, Streamlit, PostgreSQL, SQLAlchemy, Prophet, and Plotly. The implementation follows a modular software architecture that separates the frontend, backend, database, and forecasting components to improve maintainability, scalability, and code readability.

The application was developed using Visual Studio Code as the Integrated Development Environment (IDE). Version control was managed using Git and GitHub, while Render was used for cloud deployment. PostgreSQL served as the relational database for storing operational and analytical data. FastAPI was used to implement RESTful APIs, and Streamlit provided the interactive Business Intelligence dashboard.

The implementation process was divided into several independent modules:

- User Authentication
- Sales Management
- Dashboard
- Forecasting
- Reports
- Audit Logging
- Deployment

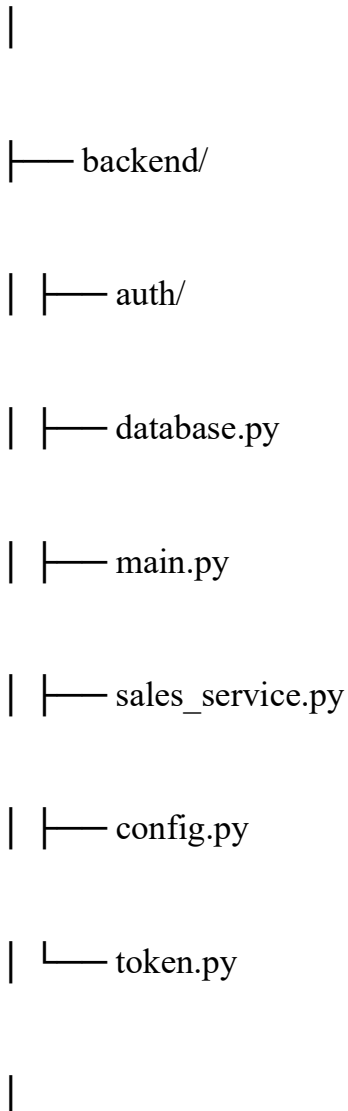
Each module was developed independently and later integrated into a unified application.

7.2 Project Folder Structure

The project follows a modular directory structure that separates source code into logical components. This organization simplifies maintenance, debugging, scalability, and future enhancements.

Figure 7.1 Project Folder Structure

AI-Powered-Sales-Forecast/



|— dashboard/

| |— Home.py

| |— sections/

| |— pages/

| |— assets/

|

|— components/

| |— filters.py

| |— styling.py

| |— charts.py

|

|— database/

| |— company_loader.py

| └─ connection.py

|

| └─ data/

| └─ Sample Forecast Data.csv

|

| └─ reports/

|

| └─ import_historical_data.py

|

| └─ requirements.txt

|

| └─ README.md

| └─ .env

Explanation

The project is divided into multiple folders based on functionality:

- **backend/** contains all FastAPI APIs and business logic.
- **dashboard/** contains the Streamlit application.
- **components/** stores reusable UI components.
- **database/** manages database connectivity.
- **data/** contains historical CSV datasets.
- **reports/** stores exported reports.
- **import_historical_data.py** imports historical sales into PostgreSQL.
- **requirements.txt** lists all Python dependencies.
- **.env** stores environment variables.

This modular organization improves code readability and maintainability.

7.3 Database Configuration

The application uses PostgreSQL as its primary relational database.

Database connectivity is established through SQLAlchemy, which simplifies communication between Python and PostgreSQL.

Database Connection Workflow

Application

↓

SQLAlchemy

↓

PostgreSQL Driver



PostgreSQL Database

The database connection is configured using an environment variable named **DATABASE_URL**.

Example:

```
DATABASE_URL = os.getenv("DATABASE_URL")
```

Using environment variables improves security by preventing sensitive credentials from being hardcoded into the source code.

The SQLAlchemy engine initializes the database connection and manages transactions throughout the application.

7.4 Backend Implementation Using FastAPI

The backend of the application was developed using the FastAPI framework.

FastAPI acts as the communication layer between the frontend dashboard and the PostgreSQL database. It processes client requests, validates incoming data, executes business logic, interacts with the database, and returns structured JSON responses.

The backend exposes several RESTful API endpoints for different application modules.

Authentication APIs

- POST /login

Sales APIs

- POST /sales
- GET /sales
- GET /sales/{bill_no}

- PUT /sales/{bill_no}
- DELETE /sales/{bill_no}
- PUT /sales/restore/{bill_no}

Dashboard APIs

- GET /dashboard
- GET /forecast

Each API endpoint performs input validation using Pydantic models before executing business logic.

FastAPI automatically generates Swagger documentation, enabling developers to test APIs directly through the browser.

7.5 User Authentication Module

Security is an essential component of enterprise software applications. The proposed system implements a secure authentication mechanism to ensure that only authorized users can access application resources.

The authentication module performs the following operations:

- User Login
- Password Verification
- JWT Generation
- Token Validation
- Role Verification

Authentication Workflow

User Login

↓

Username & Password

↓

Verify User

↓

Password Hash Check

↓

Generate JWT Token

↓

Return Token

↓

Access Dashboard

When a user enters login credentials, FastAPI retrieves the corresponding user record from PostgreSQL. The stored bcrypt hash is compared with the entered password.

If authentication succeeds, the server generates a JSON Web Token (JWT), which is returned to the client.

This token must accompany every protected API request.

7.6 JWT Authentication Implementation

JSON Web Token (JWT) provides secure stateless authentication.

Each JWT contains three components:

- Header
- Payload
- Signature

JWT Structure

Header

↓

Payload

↓

Digital Signature

The payload includes:

- Username
- User Role
- Expiration Time

When users access protected resources, FastAPI verifies the token before executing the requested operation.

If the token is invalid or expired, the server rejects the request with an authentication error.

JWT eliminates the need for maintaining server-side sessions, improving application scalability.

7.7 Role-Based Access Control (RBAC)

Role-Based Access Control ensures that users can perform only those operations permitted by their assigned role.

The application supports three user roles:

Administrator

Responsibilities:

- User Management
 - Dashboard Access
 - Sales Management
 - Reports
 - Audit Logs
 - Forecast Module
 - Restore Records
-

Manager

Responsibilities:

- Dashboard
 - Sales Management
 - Reports
 - Forecast
 - Business Analytics
-

Cashier

Responsibilities:

- Create Bills
- View Bills

- Customer Transactions

RBAC Workflow

User Login



JWT Verification



Read User Role



Permission Check



Allow / Deny Request

RBAC significantly improves system security by restricting access to sensitive operations.

7.8 Database Models Using Pydantic

To ensure data consistency, all incoming API requests are validated using Pydantic models.

Several models were created for different modules, including:

- SalesData
- BillItem
- CreateBill
- UpdateBill
- BillResponse

These models define the structure, data types, and validation rules for incoming requests.

Example validations include:

- Integer values for quantities.
- Floating-point values for prices.
- Date validation for transaction dates.
- Mandatory fields for billing operations.

By validating data before processing, the application prevents invalid or incomplete information from entering the database.

7.9 Exception Handling

The backend includes centralized exception handling to improve reliability and user experience.

Common exceptions handled include:

- Invalid login credentials
- Missing records
- Duplicate bill numbers
- Database connection failures
- Invalid request data
- Authorization failures

Whenever an exception occurs, FastAPI returns an appropriate HTTP status code and descriptive error message.

Examples include:

- **400 Bad Request**
- **401 Unauthorized**
- **404 Not Found**
- **500 Internal Server Error**

This approach improves debugging and provides meaningful feedback to users.

7.10 Chapter Summary

This section described the implementation of the application's core infrastructure. It explained the project structure, PostgreSQL database configuration, FastAPI backend architecture, user authentication mechanism, JWT implementation, Role-Based Access Control, Pydantic data validation, and exception handling. These components provide the foundation for secure communication, reliable data processing, and modular application development.

7.11 Sales Management Module

Introduction

The Sales Management Module is the core operational component of the AI-Powered Sales Forecasting and Business Intelligence Platform. It manages the complete lifecycle of restaurant billing transactions, from creating a new bill to updating existing records, performing soft deletion, restoring deleted records, and maintaining audit logs.

This module is implemented using FastAPI as the backend framework, SQLAlchemy for database communication, and PostgreSQL for persistent data storage. Each operation is exposed as a RESTful API endpoint, enabling secure interaction between the frontend dashboard and the backend services.

The major responsibilities of this module include:

- Creating new sales transactions

- Updating existing bills
- Retrieving sales records
- Soft deleting bills
- Restoring deleted bills
- Recording user activities
- Maintaining analytical datasets

The module ensures data consistency by executing all database operations within transactions, guaranteeing that either all related records are successfully updated or none are modified in the event of an error.

7.12 Create Bill Implementation

The Create Bill functionality allows cashiers and managers to generate new restaurant bills. When a bill is submitted, the application stores both the bill header and its associated item details. It also transforms transactional data into an analytical format used by the dashboard and forecasting module.

Workflow

User Creates Bill

|



Input Validation

|



Create Bill Header

|



Insert Bill Items

|



Transform Data

|



Insert into restaurant_sales

|



Audit Log Created

|



Success Response

Implementation Steps

1. Validate request using Pydantic.
2. Check whether the bill number already exists.
3. Insert bill header into restaurant_bills.
4. Retrieve the generated bill ID.
5. Insert all bill items into restaurant_bill_items.
6. Transform each item into an analytical record.
7. Store analytical records in restaurant_sales.
8. Record the operation in audit_logs.
9. Return a success response to the client.

Advantages

- Prevents duplicate bill numbers.
- Maintains normalized data.
- Automatically updates analytical datasets.
- Creates an audit trail for every transaction.

7.13 Read (Retrieve) Bill Implementation

The Read operation retrieves billing information from PostgreSQL for display on the dashboard or during editing.

The backend supports two retrieval methods:

- Retrieve all bills
- Retrieve a bill using its bill number

Workflow

Client Request

|



FastAPI Endpoint

|



SQLAlchemy Query

|



PostgreSQL

|



JSON Response

|



Dashboard Display

The retrieved data is converted into structured JSON before being returned to the frontend.

Benefits

- Fast retrieval.
- Efficient filtering.
- Dashboard compatibility.
- API integration.

7.14 Update Bill Implementation

The Update Bill feature enables managers and administrators to modify previously created bills. This operation updates both transactional records and analytical data to ensure consistency.

Workflow

Update Request

|



Locate Bill

|



Update Bill Header

|



Delete Existing Items

|



Insert Updated Items

|



Update restaurant_sales

|



Create Audit Log

|



Return Success

Implementation Process

1. Verify that the bill exists.
2. Update bill header information.
3. Remove existing bill items.
4. Insert newly modified bill items.
5. Delete previous analytical records.
6. Insert updated analytical records.
7. Create an audit log.
8. Return confirmation.

This ensures that historical analytical data always matches the latest transactional information.

Advantages

- Maintains data integrity.
- Prevents duplicate analytical records.
- Supports real-time dashboard updates.

7.15 Soft Delete Implementation

Instead of permanently removing records, the application implements **Soft Delete**.

Soft Delete marks a bill as deleted without physically removing it from the database.

Workflow

Delete Request

|



Locate Bill

|



Set is_deleted = TRUE

|



Record deleted_at

|



Store deleted_by

|



Audit Log

The following fields are updated:

- is_deleted
- deleted_at
- deleted_by

The record remains in the database but is excluded from normal dashboard queries.

Advantages

- Prevents accidental data loss.
- Allows recovery.
- Maintains historical information.
- Improves auditability.

7.16 Restore Bill Implementation

The Restore feature allows administrators to recover previously deleted bills.

Instead of recreating the bill, the system simply reverses the soft delete operation.

Workflow

Restore Request

|



Locate Deleted Bill

|



Set is_deleted = FALSE

|



Clear deleted_at

|



Clear deleted_by

|



Audit Log

|



Return Success

This operation restores the bill to active status without affecting related analytical data.

Benefits

- Fast recovery.
- No data duplication.
- Maintains transaction history.

7.17 Audit Logging Implementation

Every critical operation performed by users is recorded in the **Audit Logs** table.

The audit logging mechanism improves transparency, accountability, and security.

Logged Events

- Login
- Bill Creation
- Bill Update
- Soft Delete
- Restore
- Report Generation

Audit Workflow

User Action

|



Business Logic

|



Insert Audit Record

|



Audit Logs Table

Each audit entry stores:

- Username
- User Role
- Action
- Bill Number
- Description
- Timestamp

Benefits

- Tracks user activity.
- Detects unauthorized actions.
- Supports future compliance requirements.
- Simplifies debugging.

7.18 CRUD API Flow

The CRUD operations are exposed through RESTful APIs.

HTTP Method	Endpoint	Description
POST	/sales	Create new bill
GET	/sales	Retrieve all bills
GET	/sales/{bill_no}	Retrieve specific bill
PUT	/sales/{bill_no}	Update bill
DELETE	/sales/{bill_no}	Soft delete bill
PUT	/sales/restore/{bill_no}	Restore deleted bill

Each endpoint validates incoming data, verifies user permissions, executes business logic, updates the database, and returns an appropriate HTTP response.

7.19 Database Transactions

To ensure consistency, all major operations are executed within database transactions.

For example, during bill creation:

1. Insert bill header.
2. Insert bill items.
3. Insert analytical records.
4. Insert audit log.

If any step fails, the entire transaction is rolled back automatically.

Benefits

- Prevents partial updates.
- Maintains referential integrity.
- Ensures reliable data storage.
- Simplifies error recovery.

7.20 Exception Handling in Business Operations

The Sales Management Module includes robust exception handling to improve reliability.

Common scenarios handled include:

- Duplicate bill numbers.
- Missing records.
- Invalid input.
- Database connectivity issues.
- Authorization failures.
- Transaction rollback.

Meaningful HTTP status codes and error messages are returned to the frontend, allowing users to understand and correct issues efficiently.

7.21 Chapter Summary

This section described the implementation of the Sales Management Module, including bill creation, retrieval, updating, soft deletion, restoration, audit logging, RESTful CRUD APIs, and transaction management. These features ensure that restaurant transactions are processed securely, consistently, and efficiently while maintaining complete historical records and supporting analytical operations.

7.22 Historical Data Import Module

Introduction

To provide meaningful business insights and accurate forecasting, the system requires a substantial amount of historical sales data. Since newly created restaurant transactions alone are insufficient for long-term trend analysis, the application supports importing historical sales records from CSV files.

The historical dataset represents past restaurant transactions and serves as the foundation for dashboard visualization and sales forecasting.

The data import process was implemented using **Pandas** for data processing and **SQLAlchemy** for database insertion.

Historical Data Import Workflow

Historical CSV File

|



Read using Pandas

|



Data Cleaning



Column Mapping



Data Validation



PostgreSQL Database



Dashboard & Forecast

Import Process

The import module performs the following steps:

1. Read CSV using Pandas.
2. Verify required columns.
3. Rename dataset columns.
4. Convert date formats.
5. Handle missing values.
6. Generate derived fields.
7. Validate numerical data.
8. Insert into PostgreSQL.
9. Verify imported records.

This automated process ensures that historical data is correctly transformed into the database format required by the dashboard and forecasting modules.

Advantages

- Imports millions of records efficiently.
 - Eliminates manual database entry.
 - Supports historical analytics.
 - Improves forecast accuracy.
 - Enables long-term trend analysis.
-

7.23 Data Preprocessing

Raw business data often contains inconsistencies that must be corrected before analytical processing.

The preprocessing module prepares the imported dataset for visualization and forecasting.

Data Cleaning Operations

The application performs several preprocessing tasks.

Date Conversion

Transaction timestamps are converted into Python DateTime objects.

Missing Value Handling

Missing numerical values are replaced with default values where appropriate.

Duplicate Removal

Duplicate records are identified and removed to prevent inaccurate analytics.

Data Type Conversion

Fields such as revenue, quantity, tax, and discount are converted into appropriate numerical data types.

Feature Engineering

Additional business features are generated, including:

- Year
- Month
- Day
- Week
- Month Name

- Day Name
- Meal Period
- Weekend Flag
- Holiday Flag

These engineered features improve dashboard filtering and forecasting accuracy.

Benefits

- Improved data quality.
 - Higher forecasting accuracy.
 - Better dashboard performance.
 - Reduced processing errors.
-

7.24 Business Intelligence Dashboard

The Business Intelligence Dashboard provides restaurant managers with an interactive interface for analyzing operational performance.

Instead of static reports, the dashboard presents real-time visualizations that allow users to explore business data using interactive filters.

The dashboard was developed using **Streamlit** and **Plotly**.

Dashboard Architecture

PostgreSQL

|



FastAPI APIs

|



Streamlit

|



Interactive Dashboard

Dashboard Modules

The implemented dashboard contains several sections:

- Overview
- Analytics
- Forecast
- Reports
- Demand Analysis
- Anomaly Detection
- Model Monitoring
- AI Assistant (Disabled)

Each module provides a specific set of analytical capabilities.

7.25 Key Performance Indicators (KPIs)

To provide a quick overview of business performance, the dashboard displays several KPI cards.

The implemented KPIs include:

- Total Revenue
- Total Transactions
- Total Items Sold
- Average Bill Value
- Average Daily Sales
- Net Revenue
- Forecast Revenue
- Growth Percentage

These indicators help management evaluate restaurant performance without analyzing detailed reports.

Benefits of KPIs

- Instant business overview.
- Faster decision-making.
- Easy performance monitoring.
- Executive-level reporting.

7.26 Interactive Data Visualization

The dashboard includes multiple interactive charts that help users analyze restaurant operations.

The implemented visualizations include:

Revenue Trend Chart

Displays daily and monthly revenue growth.

Category Analysis

Shows revenue generated by each food category.

Payment Method Analysis

Visualizes customer payment preferences.

Examples:

- Cash
 - Card
 - UPI
 - Wallet
-

Meal Period Analysis

Compares business performance during:

- Breakfast
 - Lunch
 - Evening Snacks
 - Dinner
-

Top Selling Products

Ranks menu items based on sales quantity and revenue.

Weather Analysis

Displays the relationship between weather conditions and restaurant sales.

Advantages

- Interactive exploration.
 - Easy filtering.
 - Better business understanding.
 - Executive reporting.
-

7.27 Forecasting Module

The Forecasting Module predicts future restaurant revenue using the **Facebook Prophet** algorithm.

Historical sales imported into PostgreSQL serve as the training dataset.

Forecast Workflow

Historical Sales

|



Preprocessing

|



Prophet Training

|



Generate Future Dates



Predict Revenue



Forecast Visualization

Forecast Process

The forecasting module performs the following steps:

1. Retrieve historical sales.
2. Aggregate daily revenue.
3. Rename columns for Prophet (ds and y).
4. Train Prophet model.
5. Generate future dates.
6. Predict future revenue.
7. Display confidence intervals.

8. Visualize forecast.

Forecast Outputs

The dashboard displays:

- Historical Sales
- Predicted Sales
- Confidence Interval
- Future Revenue Trend

Managers can use these forecasts for:

- Inventory planning
 - Workforce scheduling
 - Budget estimation
 - Business expansion
-

7.28 Model Monitoring

To evaluate forecasting performance, the dashboard includes a Model Monitoring section.

The following evaluation metrics are displayed:

- Mean Absolute Error (MAE)
- Root Mean Square Error (RMSE)
- Mean Absolute Percentage Error (MAPE)

These metrics help determine the reliability and accuracy of the forecasting model.

The dashboard also provides a visual comparison between:

- Actual Sales
- Predicted Sales

This comparison allows users to assess forecasting performance over time.

7.29 Report Generation

The application supports exporting analytical reports for management review.

Supported report formats include:

- PDF
- Excel (XLSX)

The generated reports include:

- KPI Summary
- Revenue Charts
- Forecast Results
- Business Insights
- Transaction Summary

These reports can be shared with stakeholders for strategic planning and documentation.

7.30 AI Assistant Module

The original project architecture included an AI-powered assistant intended to answer business-related queries using historical sales data.

The AI Assistant was designed to support queries such as:

- "What was the highest selling category this month?"
- "Show the revenue trend for the last 30 days."
- "Which payment method is most frequently used?"
- "Predict next month's revenue."

During deployment, the external Large Language Model (LLM) integration was temporarily disabled because it required local AI services that are not available on the deployed cloud environment.

Instead, the deployed application displays an informational message indicating that the AI Assistant is currently disabled.

Future Enhancement

The system architecture has been designed to support future integration with cloud-based AI services such as:

- Google Gemini API
- OpenAI GPT API

This enhancement will enable conversational business analytics without modifying the overall application architecture.

7.31 Dashboard Performance Optimization

Several optimization techniques were implemented to improve dashboard responsiveness:

- Efficient SQL queries.
- Data aggregation before visualization.
- Interactive filtering.
- Optimized Plotly charts.
- Modular Streamlit components.

These optimizations reduce loading times and improve the user experience, especially when working with large historical datasets.

7.32 Chapter Summary

This section described the implementation of the analytical components of the AI-Powered Sales Forecasting and Business Intelligence Platform. It covered historical data import, preprocessing, interactive dashboard development, KPI visualization, forecasting using Prophet, model monitoring, report generation, and the AI Assistant module. Together, these components transform transactional restaurant data into actionable business insights and support data-driven decision-making.

7.33 Version Control Using Git

Introduction

Version control is an essential software engineering practice that enables developers to track changes made to source code throughout the software development lifecycle. It facilitates collaboration, maintains code history, simplifies debugging, and allows developers to revert to previous versions whenever necessary.

The AI-Powered Sales Forecasting and Business Intelligence Platform was developed using **Git**, a distributed version control system widely adopted in the software industry. Git enabled systematic tracking of project progress, feature implementation, bug fixes, and deployment-related modifications.

Throughout the development process, Git was used to:

- Track code modifications.
- Maintain project history.
- Manage multiple development iterations.
- Synchronize local and remote repositories.
- Support deployment through GitHub integration.

Git Workflow

Project Development

|



Git Init

|



Git Add

|



Git Commit

|



Git Push

|



GitHub Repository

Advantages of Git

- Distributed version control.

- Faster development.
- Easy rollback.
- Team collaboration.
- Branch management.
- Secure code history.

Git ensured that every implementation phase of the project was safely recorded, reducing the risk of accidental code loss.

7.34 GitHub Repository

GitHub served as the cloud-based repository for maintaining the project's source code and deployment pipeline.

The complete application source code, including backend services, dashboard components, database scripts, and deployment configuration, was stored in a GitHub repository.

GitHub provided:

- Source code hosting.
- Version history.
- Backup.
- Remote collaboration.
- Deployment integration.

Every completed feature was committed and synchronized with GitHub, ensuring that the latest project version remained available for deployment and future development.

Repository Structure

The repository includes:

- Backend APIs
- Dashboard

- Database Scripts
- Historical Data Import
- Documentation
- Requirements File
- Deployment Configuration

This structured organization simplifies maintenance and future enhancements.

7.35 Cloud Deployment Using Render

Introduction

Cloud deployment allows applications to be accessed through the internet without requiring local installation. The developed system was deployed using **Render**, a cloud application hosting platform supporting Python-based web applications.

Render was selected because of its:

- Free hosting tier.
 - GitHub integration.
 - Automatic deployment.
 - HTTPS support.
 - Environment variable management.
 - Continuous deployment.
-

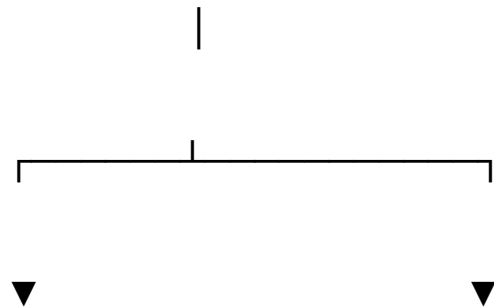
Deployment Architecture

GitHub Repository

|



Render Platform



FastAPI API Streamlit Dashboard



Public Web URLs

Backend Deployment

The FastAPI application was deployed as a Render Web Service.

Deployment steps included:

1. Connect GitHub repository.
2. Configure Python runtime.
3. Install project dependencies.
4. Configure environment variables.
5. Deploy using Uvicorn.
6. Generate public API URL.

The deployed backend provides secure REST APIs accessible over HTTPS.

Dashboard Deployment

The Streamlit dashboard was deployed separately on Render.

Deployment involved:

1. Creating a second Render Web Service.
2. Installing dependencies from requirements.txt.
3. Configuring the startup command.
4. Publishing the dashboard URL.

The deployed dashboard allows users to interact with the application using a standard web browser.

Deployment URLs

The project was successfully deployed using two independent services:

FastAPI Backend

Provides secure RESTful API endpoints for authentication, sales management, dashboard data, forecasting, and reporting.

Streamlit Dashboard

Provides the interactive Business Intelligence interface used by restaurant managers.

7.36 Environment Variable Configuration

Sensitive application credentials should never be hardcoded within source code.

The project stores confidential information using environment variables.

Examples include:

- DATABASE_URL
- SECRET_KEY

During deployment, these variables were configured within the Render dashboard.

Benefits

- Improved security.
- Simplified deployment.
- Easy environment switching.
- Prevents credential leakage.

This approach follows modern DevOps and cloud security best practices.

7.37 Challenges Faced During Development

The development of the AI-Powered Sales Forecasting and Business Intelligence Platform presented several technical challenges.

Challenge 1: Historical Data Import

The historical dataset contained millions of records, requiring efficient preprocessing and optimized insertion into PostgreSQL.

Solution

Pandas was used for data cleaning and transformation, while SQLAlchemy managed efficient database insertion.

Challenge 2: CRUD Synchronization

Updating restaurant bills required synchronization between transactional and analytical tables.

Solution

Database transactions were used to ensure that updates remained consistent across all related tables.

Challenge 3: Forecast Accuracy

Forecasting required sufficient historical data and proper preprocessing to produce meaningful predictions.

Solution

Historical restaurant sales were imported and cleaned before training the Prophet model.

Challenge 4: Authentication

Secure access control was required to prevent unauthorized operations.

Solution

JWT Authentication and Role-Based Access Control (RBAC) were implemented.

Challenge 5: Deployment

The AI Assistant relied on a locally hosted Large Language Model (LLM), which was not compatible with the cloud deployment environment.

Solution

The AI Assistant was temporarily disabled in the deployed version while preserving the system architecture for future cloud-based AI integration.

Challenge 6: Dashboard Performance

Rendering large analytical datasets caused increased loading times.

Solution

Optimized SQL queries, data aggregation, and interactive filtering improved dashboard responsiveness.

7.38 Future Scalability

The modular architecture was designed to support future enhancements without major architectural modifications.

Potential enhancements include:

- Cloud PostgreSQL database.
- Google Gemini integration.
- OpenAI GPT integration.
- Inventory Management.
- Supplier Management.
- Customer Loyalty System.
- Mobile Application.
- Multi-Branch Restaurant Management.
- Real-time Analytics.
- Automated Email Reports.
- Voice-based Business Assistant.
- Inventory Forecasting.
- Demand Prediction using Deep Learning.
- Multi-language Dashboard.
- Docker-based deployment.
- CI/CD automation.

These enhancements can be integrated while preserving the existing modular architecture.

7.39 Chapter Summary

This chapter described the complete implementation of the AI-Powered Sales Forecasting and Business Intelligence Platform. It explained the modular project structure, backend development using FastAPI, frontend implementation with Streamlit, secure authentication using JWT, Role-Based Access Control, CRUD operations, historical data import, business intelligence dashboards, Prophet-based forecasting, report generation, version control using Git, cloud deployment through Render, and practical challenges encountered during

development. The implementation demonstrates the successful integration of modern software engineering practices, predictive analytics, and business intelligence technologies into a unified, scalable application suitable for restaurant management.

CHAPTER 8

SYSTEM TESTING

8.1 Introduction

System testing is a crucial phase in the Software Development Life Cycle (SDLC) that ensures the developed software meets its functional and non-functional requirements before deployment. The objective of testing is to identify defects, verify that all modules work as intended, and ensure that the integrated system operates reliably under expected conditions.

The AI-Powered Sales Forecasting and Business Intelligence Platform underwent comprehensive testing throughout its development. Individual modules were tested independently before integration, and the complete application was validated using real restaurant sales data. Testing included authentication, CRUD operations, dashboard visualization, forecasting, report generation, historical data processing, and cloud deployment.

The primary goals of testing were:

- Verify correctness of all implemented features.
- Ensure secure authentication and authorization.
- Validate database operations.
- Confirm forecasting functionality.
- Test dashboard performance.
- Ensure API reliability.
- Verify report generation.
- Validate deployment.

8.2 Testing Methodology

The project followed a structured testing methodology consisting of multiple testing levels.

Requirements

|



Unit Testing

|



Integration Testing

|



System Testing

|



User Acceptance Testing

|



Deployment Verification

Each testing phase focused on a specific aspect of the application to ensure overall system reliability.

8.3 Unit Testing

Unit testing verifies the correctness of individual software components independently.

Each backend function and dashboard module was tested separately before integration.

Modules Tested

- Authentication
- JWT Token Generation
- Password Verification
- CRUD Operations
- Database Functions
- Forecast Module
- Dashboard Components
- Report Generation
- Historical Data Import

Objectives

- Validate individual functions.
- Detect programming errors.
- Ensure correct input/output behavior.

Result

All implemented modules produced the expected outputs after correcting identified issues during development.

8.4 Integration Testing

Integration testing verifies communication between multiple software modules.

The following integrations were tested:

- Streamlit ↔ FastAPI
- FastAPI ↔ PostgreSQL
- FastAPI ↔ Prophet
- Dashboard ↔ Database
- Dashboard ↔ Reports
- Authentication ↔ APIs

Objectives

- Verify data flow.
- Validate API communication.
- Ensure module compatibility.

Result

Successful integration was achieved between all major application components.

8.5 System Testing

System testing evaluates the complete application as a single integrated system.

The following functionalities were verified:

- User Login
- Dashboard Navigation
- CRUD Operations

- Forecast Generation
- Historical Analysis
- Reports
- Audit Logging
- Soft Delete
- Restore
- Cloud Deployment

Result

The complete application functioned successfully without critical failures.

8.6 API Testing

All RESTful APIs developed using FastAPI were tested using Swagger UI.

APIs Tested

API	Status
Login API	Pass
Create Bill API	Pass
Get Bills API	Pass
Update Bill API	Pass
Delete Bill API	Pass
Restore Bill API	Pass
Forecast API	Pass
Dashboard API	Pass

Testing Objectives

- Validate request format.
- Verify HTTP status codes.

- Ensure JSON responses.
- Test authorization.

Result

All APIs returned expected responses.

8.7 Authentication Testing

Authentication testing ensured secure user access.

Test Cases

- Valid Login
- Invalid Username
- Invalid Password
- Expired JWT
- Missing Token
- Unauthorized Access

Result

The authentication system successfully prevented unauthorized access while allowing authenticated users to access protected resources.

8.8 CRUD Testing

CRUD operations were tested individually.

Create

- Create new bill.
- Verify database insertion.
- Verify dashboard update.

Status: Pass

Read

- Retrieve all bills.
- Retrieve bill by number.

Status: Pass

Update

- Modify bill.
- Verify updated records.

Status: Pass

Delete

- Soft delete bill.
- Verify record visibility.

Status: Pass

Restore

- Restore deleted bill.
- Verify dashboard update.

Status: Pass

8.9 Dashboard Testing

The dashboard was tested using historical sales data.

Dashboard Components Tested

- KPI Cards
- Revenue Chart
- Category Analysis

- Payment Analysis
- Meal Period Analysis
- Forecast Charts
- Filters
- Reports

Result

All visualizations displayed accurate information.

Interactive filters updated charts dynamically without errors.

8.10 Forecast Testing

Forecast testing verified the accuracy and stability of the Prophet forecasting module.

Validation Steps

- Load historical data.
- Train Prophet model.
- Generate predictions.
- Display forecast graph.

Evaluation Metrics

- MAE
- RMSE
- MAPE

Result

The forecasting model successfully generated future revenue predictions based on historical restaurant sales.

8.11 Historical Data Import Testing

The historical dataset imported from CSV was validated before use.

Tests Performed

- File loading
- Column validation
- Date conversion
- Missing values
- Database insertion

Result

Historical sales records were successfully imported into PostgreSQL and became immediately available for dashboard analytics and forecasting.

8.12 Report Generation Testing

Both PDF and Excel report generation modules were tested.

Reports Generated

- KPI Summary
- Revenue Analysis
- Forecast Report
- Business Insights

Result

Reports were generated successfully and contained accurate information.

8.13 Deployment Testing

The application was tested after deployment on Render.

Backend Testing

- API accessibility
- Authentication

- CRUD APIs
- HTTPS

Dashboard Testing

- Dashboard loading
- Navigation
- Charts
- Forecast

Result

Both the FastAPI backend and Streamlit dashboard were successfully deployed and accessible through public URLs.

8.14 User Acceptance Testing (UAT)

User Acceptance Testing was conducted to ensure that the application satisfied business requirements.

Scenarios

- Login
- Create Bill
- Update Bill
- Dashboard Analysis
- Forecast Generation
- Report Download
- Restore Bill

Outcome

The application fulfilled the expected operational requirements and provided an intuitive interface for restaurant management.

8.15 Test Case Summary

Test ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	User Login	Login successful	Login successful	Pass
TC-02	Invalid Login	Error message	Correct error displayed	Pass
TC-03	Create Bill	Bill stored	Bill stored	Pass
TC-04	Update Bill	Bill updated	Updated successfully	Pass
TC-05	Soft Delete	Bill hidden	Hidden successfully	Pass
TC-06	Restore Bill	Bill restored	Restored successfully	Pass
TC-07	Dashboard Load	Charts displayed	Charts displayed	Pass
TC-08	Forecast	Revenue predicted	Prediction generated	Pass
TC-09	Report Export	PDF/Excel created	Created successfully	Pass
TC-10	API Access	JSON response	Valid response	Pass

8.16 Bug Fixes During Development

Several issues were identified and resolved during development.

Issue	Solution
Database import errors	Corrected column mapping and validation
JWT authentication errors	Updated token verification logic
CRUD synchronization issues	Used database transactions
Dashboard filter duplication	Assigned unique widget keys
Forecast data mismatch	Improved preprocessing pipeline

Issue

Solution

Deployment environment issues Configured environment variables

AI dependency on local LLM Disabled AI module in cloud deployment

These corrections improved the stability and reliability of the application.

8.17 Testing Summary

Comprehensive testing confirmed that the AI-Powered Sales Forecasting and Business Intelligence Platform operates reliably and meets its functional requirements. All critical modules, including authentication, CRUD operations, dashboard visualization, forecasting, reporting, historical data processing, and deployment, were successfully validated. The testing process ensured data integrity, secure access control, accurate forecasting, and smooth interaction between the frontend and backend components, resulting in a stable application suitable for demonstration and future enhancements.

CHAPTER 9

RESULTS AND DISCUSSION

9.1 Introduction

The Results and Discussion chapter presents the outcomes obtained after implementing the AI-Powered Sales Forecasting and Business Intelligence Platform. It demonstrates how the developed application successfully integrates restaurant billing, business intelligence, historical data analysis, predictive forecasting, secure authentication, and cloud deployment into a unified system.

The application was tested using a large historical restaurant sales dataset imported into PostgreSQL along with newly created transaction records. The generated dashboards, reports, and forecasts confirmed that the system functions correctly and provides meaningful business insights.

The implementation achieved all the objectives defined during the requirement analysis phase and demonstrated that the proposed solution can effectively support restaurant management through data-driven decision-making.

9.2 Login Module Results

The authentication module was successfully implemented using JSON Web Token (JWT) authentication and Role-Based Access Control (RBAC).

Users can securely log into the system using their registered credentials. Upon successful authentication, a JWT token is generated and used to authorize access to protected APIs.

Features Verified

- Secure Login
- Password Encryption
- JWT Token Generation

- Token Verification
- Role-Based Access Control

Result

The login process successfully authenticated authorized users while preventing unauthorized access.

Insert Figure 9.1: Login Page Screenshot

9.3 Dashboard Results

The Business Intelligence Dashboard successfully visualized restaurant performance through interactive charts and Key Performance Indicators (KPIs).

The dashboard dynamically retrieved data from PostgreSQL and displayed meaningful business insights using Plotly visualizations.

The implemented dashboard includes:

- Revenue Dashboard
- Sales Analytics
- Payment Analysis
- Category Analysis
- Forecast Dashboard
- Reports
- Demand Analysis
- Model Monitoring

Dashboard Outcome

Managers were able to monitor restaurant performance without manually generating reports.

Interactive filters enabled users to analyze sales based on:

- Date
- Category

- Payment Method
- Meal Period
- Weather
- Service Type

Insert Figure 9.2: Dashboard Home Screen

9.4 KPI Results

The dashboard displayed several Key Performance Indicators that summarize overall business performance.

The KPIs implemented include:

- Total Revenue
- Net Revenue
- Total Transactions
- Average Bill Value
- Total Items Sold
- Growth Percentage
- Forecast Revenue

These KPIs enabled management to evaluate business performance instantly.

Discussion

Displaying KPIs at the top of the dashboard significantly reduced the time required for managers to understand restaurant performance.

Insert Figure 9.3: KPI Cards

9.5 Revenue Analysis Results

The Revenue Analysis module successfully visualized revenue trends over time.

The dashboard generated interactive graphs showing:

- Daily Revenue
- Monthly Revenue
- Yearly Revenue

The imported historical dataset enabled long-term trend analysis.

Observations

- Revenue fluctuations became clearly visible.
- Seasonal peaks were easily identified.
- Managers could compare historical performance with current sales.

Benefits

- Better financial planning.
- Easier trend identification.
- Improved strategic decision-making.

Insert Figure 9.4: Revenue Trend Chart

9.6 Category Analysis Results

Category Analysis identified the contribution of each food category to overall restaurant revenue.

The dashboard displayed interactive bar charts ranking food categories based on:

- Revenue
- Quantity Sold
- Number of Transactions

Discussion

The visualization enabled management to identify:

- Best-selling categories.
- Low-performing categories.

- Customer purchasing preferences.

This information can be used to optimize menu planning and promotional strategies.

Insert Figure 9.5: Category Analysis Chart

9.7 Payment Method Analysis Results

The Payment Analysis module displayed customer payment preferences.

Supported payment methods include:

- Cash
- Card
- UPI
- Wallet

Discussion

Understanding payment distribution enables restaurant management to:

- Improve payment infrastructure.
- Promote preferred payment methods.
- Analyze customer behavior.

Insert Figure 9.6: Payment Method Chart

9.8 Historical Data Analysis Results

One of the major achievements of the project was the successful integration of historical restaurant sales.

The historical dataset contained a large number of restaurant transactions, enabling meaningful business analysis and forecasting.

Historical Data Benefits

- Long-term trend analysis.
- Better dashboard visualization.

- Improved forecast accuracy.
- Rich analytical insights.

Result

The imported data significantly improved the usefulness of the Business Intelligence dashboard.

Insert Figure 9.7: Historical Sales Dashboard

9.9 Sales Forecast Results

The Prophet forecasting module successfully generated future restaurant revenue predictions.

Historical sales data was used to train the forecasting model.

The model generated:

- Future Revenue
- Confidence Interval
- Trend Analysis

Discussion

The generated forecasts provide valuable support for:

- Inventory Planning
- Employee Scheduling
- Revenue Estimation
- Business Expansion

Forecasting enables proactive decision-making rather than relying solely on historical reports.

Insert Figure 9.8: Forecast Graph

9.10 CRUD Operation Results

The CRUD module successfully supported the complete lifecycle of restaurant bills.

Operations tested:

- Create
- Read
- Update
- Soft Delete
- Restore

Discussion

All CRUD operations were executed successfully while maintaining consistency across transactional and analytical tables.

The Soft Delete feature prevented accidental data loss, while the Restore feature allowed recovery of deleted records.

9.11 Audit Logging Results

Every critical user activity was successfully recorded in the Audit Logs table.

Logged activities include:

- Login
- Bill Creation
- Bill Update
- Delete
- Restore

Benefits

- Improved accountability.
- Better security.
- Activity monitoring.
- Future compliance support.

Insert Figure 9.9: Audit Logs Screen

9.12 Report Generation Results

The application successfully generated business reports in:

- PDF Format
- Excel Format

Generated reports include:

- KPI Summary
- Revenue Analysis
- Forecast Results
- Business Insights

Discussion

Managers can download reports for offline analysis and decision-making.

Insert Figure 9.10: Generated Report

9.13 Deployment Results

The application was successfully deployed on the Render cloud platform.

Deployment included:

FastAPI Backend

The backend APIs became accessible through a public HTTPS endpoint.

Streamlit Dashboard

The interactive dashboard was successfully deployed as an independent web application.

Outcome

The deployed application demonstrates production readiness and allows remote access through a standard web browser.

9.14 Performance Evaluation

The application performed efficiently during testing.

Performance Highlights

- Fast dashboard loading.
- Efficient database queries.
- Smooth CRUD operations.
- Reliable forecasting.
- Secure authentication.
- Stable deployment.

Database Performance

PostgreSQL successfully managed historical restaurant sales and transactional records while maintaining good query performance.

9.15 Comparison of Objectives and Achievements

Project Objective	Achievement
Secure Authentication	Achieved
Restaurant Billing	Achieved
CRUD Operations	Achieved
Historical Data Import	Achieved
Business Intelligence Dashboard	Achieved
Interactive Charts	Achieved
Prophet Forecasting	Achieved
Report Generation	Achieved

Project Objective	Achievement
Audit Logging	Achieved
Cloud Deployment	Achieved

The comparison confirms that all primary objectives defined at the beginning of the project were successfully accomplished.

9.16 Overall Discussion

The AI-Powered Sales Forecasting and Business Intelligence Platform successfully demonstrates the practical integration of modern software engineering techniques with predictive analytics and business intelligence.

The system not only automates restaurant billing operations but also transforms transactional data into meaningful business insights through interactive dashboards and forecasting. Secure authentication, audit logging, historical data analysis, and cloud deployment further enhance the reliability and usability of the application.

Although the conversational AI assistant was disabled in the deployed version due to cloud deployment constraints, the application architecture remains extensible for future integration with cloud-based Large Language Models such as Google Gemini or OpenAI.

The project achieved its primary goal of providing restaurant management with a scalable, secure, and intelligent decision-support system capable of improving operational efficiency and supporting data-driven business decisions.

9.17 Chapter Summary

This chapter presented the results obtained after implementing the AI-Powered Sales Forecasting and Business Intelligence Platform. It discussed the successful implementation of secure authentication, interactive dashboards, historical data analysis, sales forecasting, CRUD operations, audit logging, report generation, and cloud deployment. The generated results demonstrate that the developed system effectively meets its objectives and provides valuable

business insights for restaurant management through predictive analytics and Business Intelligence.

Excellent. This is the **final chapter** of your internship report. It summarizes the work completed, highlights the achievements, discusses current limitations, and outlines future enhancements. A well-written conclusion leaves a strong impression on the evaluator.

CHAPTER 10

CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Introduction

The restaurant industry has become increasingly dependent on digital technologies to improve operational efficiency, customer satisfaction, and business profitability. Traditional restaurant management systems primarily focus on recording transactions and generating basic reports, which provide limited support for strategic decision-making. The integration of Business Intelligence (BI), Predictive Analytics, and Artificial Intelligence (AI) enables organizations to transform historical operational data into actionable business insights.

The AI-Powered Sales Forecasting and Business Intelligence Platform developed during this internship addresses these challenges by combining secure transaction management, interactive dashboards, predictive forecasting, and cloud deployment into a unified software solution. The project demonstrates how modern software engineering practices and analytical technologies can be applied to improve restaurant management and support data-driven decision-making.

10.2 Project Summary

The primary objective of this project was to develop an intelligent restaurant management platform capable of integrating operational management with Business Intelligence and sales forecasting.

The application was developed using a modern technology stack consisting of:

- Python
- FastAPI
- Streamlit
- PostgreSQL

- SQLAlchemy
- Prophet
- Plotly
- Pandas
- JWT Authentication
- bcrypt Password Hashing
- Git & GitHub
- Render Cloud Platform

The completed application provides:

- Secure User Authentication
- Role-Based Access Control
- Restaurant Billing
- Sales Management
- CRUD Operations
- Soft Delete and Restore
- Audit Logging
- Historical Sales Analysis
- Interactive Business Intelligence Dashboard
- Prophet-based Sales Forecasting
- Report Generation
- Cloud Deployment

The system successfully combines operational data management with predictive analytics, enabling restaurant managers to monitor business performance and make informed decisions based on historical trends and future forecasts.

10.3 Objectives Achieved

The objectives defined during the planning stage were successfully accomplished.

Objective 1

Develop a secure restaurant billing system.

Status: Achieved

Objective 2

Implement Role-Based Authentication.

Status: Achieved

Objective 3

Store restaurant data using PostgreSQL.

Status: Achieved

Objective 4

Develop REST APIs using FastAPI.

Status: Achieved

Objective 5

Create an interactive Business Intelligence Dashboard.

Status: Achieved

Objective 6

Analyze historical restaurant sales.

Status: Achieved

Objective 7

Implement sales forecasting using Prophet.

Status: Achieved

Objective 8

Generate downloadable business reports.

Status: Achieved

Objective 9

Implement audit logging.

Status: Achieved

Objective 10

Deploy the application on the cloud.

Status: Achieved

10.4 Project Outcomes

The project successfully demonstrated that integrating modern software technologies with Business Intelligence significantly improves restaurant management.

The developed system enables restaurant owners and managers to:

- Monitor restaurant performance in real time.
- Analyze historical sales trends.
- Identify top-performing products.
- Track revenue growth.

- Evaluate customer payment preferences.
- Predict future sales.
- Generate management reports.
- Maintain secure transaction records.
- Restore accidentally deleted records.
- Monitor user activities.

The cloud deployment further demonstrates that the application is production-ready and accessible through the internet.

10.5 Benefits of the Developed System

The AI-Powered Sales Forecasting and Business Intelligence Platform offers several operational and strategic benefits.

Operational Benefits

- Automated billing.
 - Reduced manual effort.
 - Centralized database.
 - Faster transaction processing.
 - Secure authentication.
 - Audit logging.
-

Analytical Benefits

- Interactive dashboards.
- Historical trend analysis.
- Revenue monitoring.
- Product performance analysis.
- Customer behavior analysis.

- Business KPI visualization.
-

Forecasting Benefits

- Future revenue prediction.
 - Inventory planning.
 - Employee scheduling.
 - Business growth planning.
 - Demand estimation.
-

Technical Benefits

- Modular architecture.
 - RESTful APIs.
 - Cloud deployment.
 - High scalability.
 - Easy maintenance.
-

10.6 Limitations of the Current System

Although the developed application successfully satisfies the project objectives, several limitations remain.

Local Database During Development

The application was developed using a locally hosted PostgreSQL database. A managed cloud database would improve accessibility and scalability.

AI Assistant

The AI Assistant was temporarily disabled in the deployed version because the original implementation relied on a locally hosted Large Language Model (LLM). Cloud-based AI integration requires external API services.

Single Branch Support

The current system manages data for a single restaurant location. Multi-branch functionality has not yet been implemented.

Inventory Integration

Inventory and supplier management are outside the scope of the current implementation.

Mobile Accessibility

The application is optimized for desktop browsers. A dedicated mobile application has not yet been developed.

Real-Time Notifications

The system currently does not provide automated email or SMS notifications for critical business events.

10.7 Future Enhancements

The modular architecture allows the application to be extended with additional enterprise-level features.

Cloud Database Integration

Migrate PostgreSQL to a managed cloud database such as Supabase or Render PostgreSQL to improve availability and scalability.

AI-Powered Business Assistant

Integrate cloud-based Large Language Models such as Google Gemini or OpenAI GPT to enable conversational business analytics and intelligent recommendations.

Inventory Management

Implement stock monitoring, inventory forecasting, supplier management, and automatic stock alerts.

Multi-Branch Restaurant Management

Support multiple restaurant locations with centralized reporting and branch-wise analytics.

Customer Relationship Management (CRM)

Develop customer profiles, loyalty programs, purchase history, and personalized promotional campaigns.

Mobile Application

Develop Android and iOS applications to allow managers to monitor business performance remotely.

Real-Time Notification System

Integrate email, SMS, or messaging services to notify managers about important business events, forecast alerts, or unusual sales patterns.

Advanced Machine Learning Models

Compare Prophet with additional forecasting models such as:

- XGBoost
- LSTM Neural Networks
- Random Forest
- ARIMA

- SARIMA

This comparison could improve forecasting accuracy for different business scenarios.

Docker and CI/CD

Containerize the application using Docker and implement Continuous Integration/Continuous Deployment (CI/CD) pipelines for automated testing and deployment.

Business Recommendation Engine

Develop a recommendation system capable of suggesting:

- Menu optimization
- Pricing strategies
- Promotional campaigns
- Inventory adjustments

based on historical sales and forecast results.

10.8 Conclusion

The AI-Powered Sales Forecasting and Business Intelligence Platform successfully demonstrates how modern software engineering, Business Intelligence, and predictive analytics can be integrated to improve restaurant management.

The developed system not only automates restaurant billing but also transforms transactional data into valuable business insights through interactive dashboards and accurate sales forecasting. The implementation of FastAPI, Streamlit, PostgreSQL, SQLAlchemy, and Prophet provides a scalable and maintainable architecture capable of supporting future enhancements.

The inclusion of secure authentication, Role-Based Access Control, audit logging, historical data analysis, cloud deployment, and predictive forecasting

enhances both operational efficiency and strategic decision-making. Managers can monitor key performance indicators, evaluate sales trends, forecast future demand, and make informed business decisions using a single integrated platform.

The successful deployment of the application on the Render cloud platform further demonstrates its practical applicability and production readiness. Although certain advanced features, such as cloud-based AI assistants and inventory management, remain future enhancements, the current implementation provides a strong foundation for intelligent restaurant management systems.

Overall, the project successfully achieved its objectives and showcases the practical application of Business Intelligence, predictive analytics, cloud computing, and modern web technologies in solving real-world business problems. It represents a scalable, secure, and extensible solution capable of supporting future innovation in the restaurant and hospitality industry.

APPENDIX

APPENDIX A

API DOCUMENTATION

A.1 Introduction

The AI-Powered Sales Forecasting and Business Intelligence Platform exposes a set of RESTful APIs developed using **FastAPI**. These APIs facilitate secure communication between the Streamlit dashboard and the PostgreSQL database while supporting restaurant billing, authentication, forecasting, and reporting.

All protected APIs require authentication using a JSON Web Token (JWT). Responses are returned in JSON format with appropriate HTTP status codes.

A.2 Authentication API

Login

Endpoint

POST /login

Description

Authenticates the user and generates a JWT access token.

Request Body

```
{  
  "username": "admin",  
  "password": "admin123"  
}
```

Success Response

```
{  
  "access_token": "eyJhbGc...",
```



```
"token_type": "bearer",  
"role": "Admin"  
}
```

Status Codes

Code	Description
200	Login Successful
401	Invalid Username or Password
500	Server Error

A.3 Create Sales API

Endpoint

POST /sales

Description

Creates a new restaurant bill.

Request Body

```
{  
  "bill_no":101,  
  "customer_name":"John",  
  "payment_method":"Cash",  
  "service_type":"Dine-In",  
  "items":[]  
}
```

Success Response

```
{  
  "message":"Bill Created Successfully"
```

```
}
```

A.4 Get All Sales

Endpoint

GET /sales

Description

Returns all restaurant bills.

Response

```
[  
  {  
    "bill_no":101,  
    "customer_name":"John"  
  }  
]
```

A.5 Get Bill by Bill Number

Endpoint

GET /sales/{bill_no}

Example

GET /sales/101

Description

Returns complete bill details.

A.6 Update Bill

Endpoint

PUT /sales/{bill_no}

Description

Updates an existing bill.

Response

```
{  
  "message": "Bill updated successfully"  
}
```

A.7 Soft Delete Bill**Endpoint**

DELETE /sales/{bill_no}

Description

Marks a bill as deleted without permanently removing it from the database.

A.8 Restore Bill**Endpoint**

PUT /sales/restore/{bill_no}

Description

Restores a previously soft-deleted bill.

A.9 Dashboard API**Endpoint**

GET /dashboard

Returns dashboard analytics.

A.10 Forecast API**Endpoint**

GET /forecast

Returns Prophet forecasting results.

A.11 Authentication Flow

User

↓

Login API

↓

JWT Generated

↓

Token Returned

↓

Protected APIs

A.12 HTTP Status Codes Used

Code Meaning

200 Success

Code Meaning

201 Created

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

A.13 API Security

The backend implements:

- JWT Authentication
 - bcrypt Password Hashing
 - Role-Based Access Control
 - Protected Endpoints
 - Request Validation using Pydantic
-

APPENDIX B**DATABASE SCHEMA****B.1 Users Table**

Column	Data Type	Description
user_id	BIGINT	Primary Key
username	VARCHAR	Username
password	VARCHAR	Encrypted Password

Column	Data Type	Description
role	VARCHAR	Admin/Manager/Cashier
created_at	TIMESTAMP	Account Creation

B.2 Restaurant Bills Table

Column	Description
bill_id	Primary Key
bill_no	Bill Number
bill_date	Bill Date
customer_name	Customer
service_type	Dine-In/Takeaway
payment_method	Cash/Card
gross_sales	Gross Sales
discount_amount	Discount
tax_amount	GST
delivery_charge	Delivery
total_billed	Final Amount
tips	Tips
net_revenue	Revenue
is_deleted	Soft Delete Flag
deleted_at	Deleted Timestamp
deleted_by	Deleted User

B.3 Restaurant Bill Items

Column	Description
bill_item_id	Primary Key
bill_id	Foreign Key
item_name	Food Item
category	Category
menu_group	Menu Group
quantity	Quantity
item_rate	Unit Price
discount_amount	Discount

B.4 Restaurant Sales

Column	Description
Date	Sale Date
BillNo	Bill Number
Category	Food Category
Item_Name	Product
Quantity	Quantity Sold
Gross_Sales	Revenue
Net_Revenue	Final Revenue
Payment_Method	Payment
Meal_Period	Breakfast/Lunch

Column	Description
Weather	Weather
Is_Holiday	Holiday Flag
Is_Weekend	Weekend Flag

B.5 Audit Logs

Column	Description
audit_id	Primary Key
username	Username
role	Role
action	CRUD Action
bill_no	Bill Number
description	Activity
timestamp	Time

B.6 Entity Relationship

Users

|



Audit Logs

Restaurant Bills

|



Restaurant Bill Items



Restaurant Sales

APPENDIX C

INSTALLATION GUIDE

Software Requirements

- Windows 10/11
- Python 3.13+
- PostgreSQL
- VS Code
- Git

Step 1

Clone Repository

```
git clone <repository-url>
```

Step 2

Open Project

```
cd AI-Powered-Sales-Forecast
```

Step 3

Create Virtual Environment

```
python -m venv venv
```

Step 4

Activate Virtual Environment

Windows

```
venv\Scripts\activate
```

Linux

```
source venv/bin/activate
```

Step 5

Install Dependencies

```
pip install -r requirements.txt
```

Step 6

Configure Environment Variables

Create .env

```
DATABASE_URL=postgresql://username:password@localhost:5432/database
```

```
SECRET_KEY=your_secret_key
```

Step 7

Create PostgreSQL Database

```
CREATE DATABASE sales_forecast_db;
```

Step 8

Execute SQL Scripts

Run all database schema SQL files.

Step 9

Import Historical Data

```
python import_historical_data.py
```

Step 10

Run FastAPI

```
uvicorn backend.main:app --reload
```

Step 11

Run Streamlit

```
streamlit run dashboard/Home.py
```

APPENDIX D

DEPLOYMENT GUIDE

Backend Deployment

Platform

Render

Start Command

```
uvicorn backend.main:app --host 0.0.0.0 --port $PORT
```

Dashboard Deployment

Start Command

```
streamlit run dashboard/Home.py --server.port=$PORT --server.address=0.0.0.0
```

Environment Variables

DATABASE_URL=...

SECRET_KEY=...

GitHub Integration

Connect Render to GitHub repository.

Automatic deployment starts after every push.

Deployment Verification

Verify:

- Login
 - Dashboard
 - CRUD
 - Forecast
 - Reports
-

APPENDIX E

USER MANUAL

Step 1

Open Dashboard URL.

Step 2

Login.

Step 3

Dashboard Home

Displays:

- KPIs
 - Revenue
 - Charts
-

Step 4

Sales Module

- Create Bill
 - Update Bill
 - Delete
 - Restore
-

Step 5

Forecast Module

Select forecast period.

View prediction graph.

Step 6

Reports

Download:

- PDF
 - Excel
-

Step 7

Logout

Securely exit application.

APPENDIX F

SAMPLE SCREENSHOTS

Figure F.1

Dashboard Home

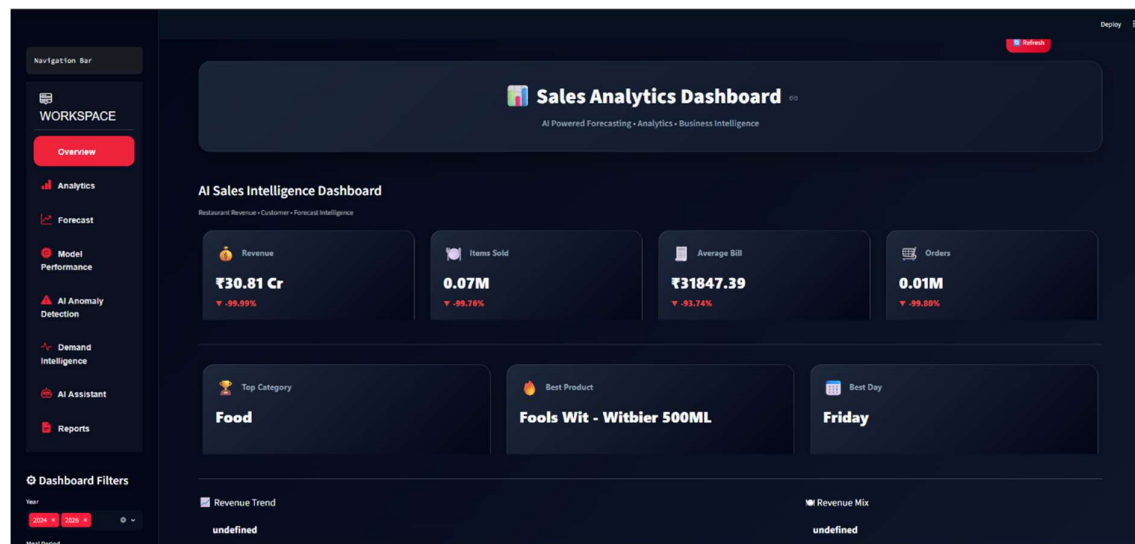


Figure F.2

KPI Cards

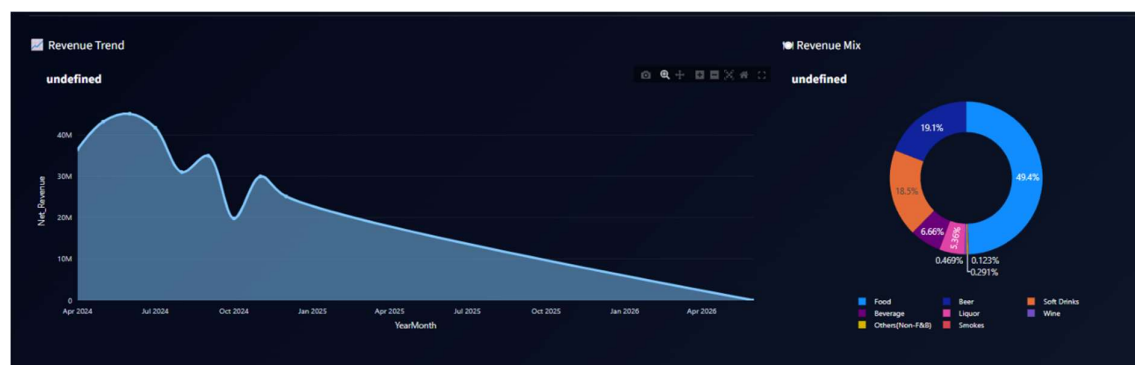
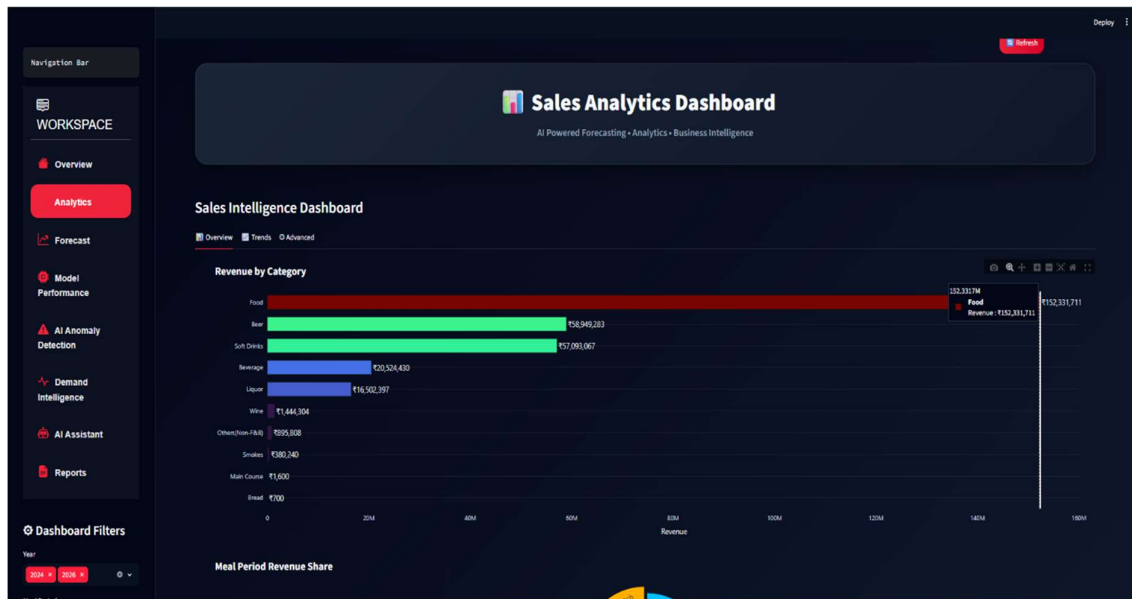


Figure F.3

Analytics Page

**Figure F.4**

Forecast Page



Figure F.5

Model Monitoring

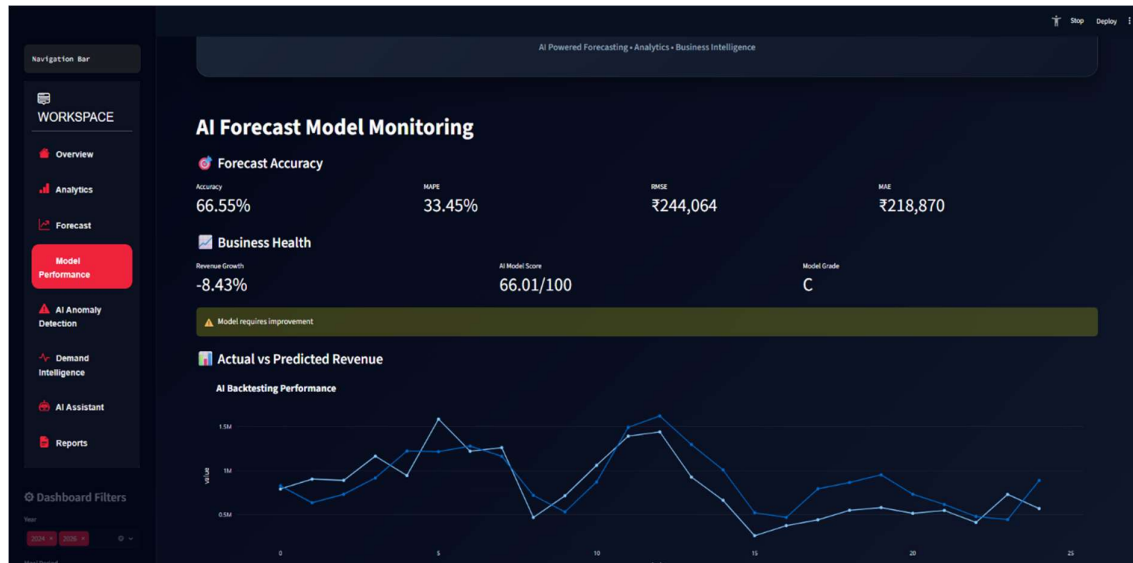


Figure F.6

AI Anomaly Detection

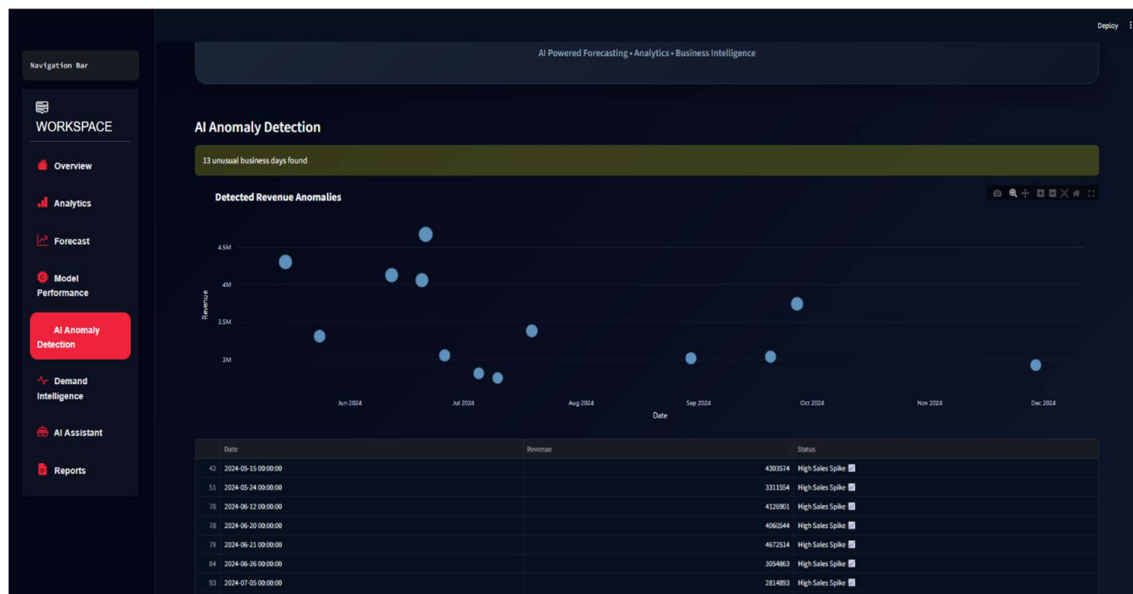


Figure F.7

Demand Intelligence

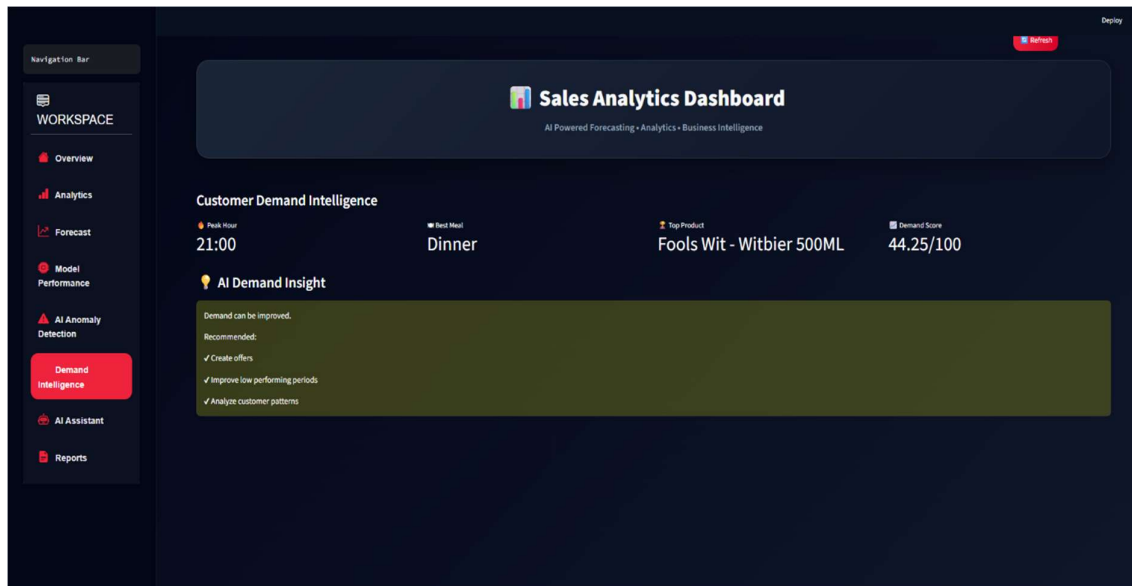


Figure F.8

AI Assiatance

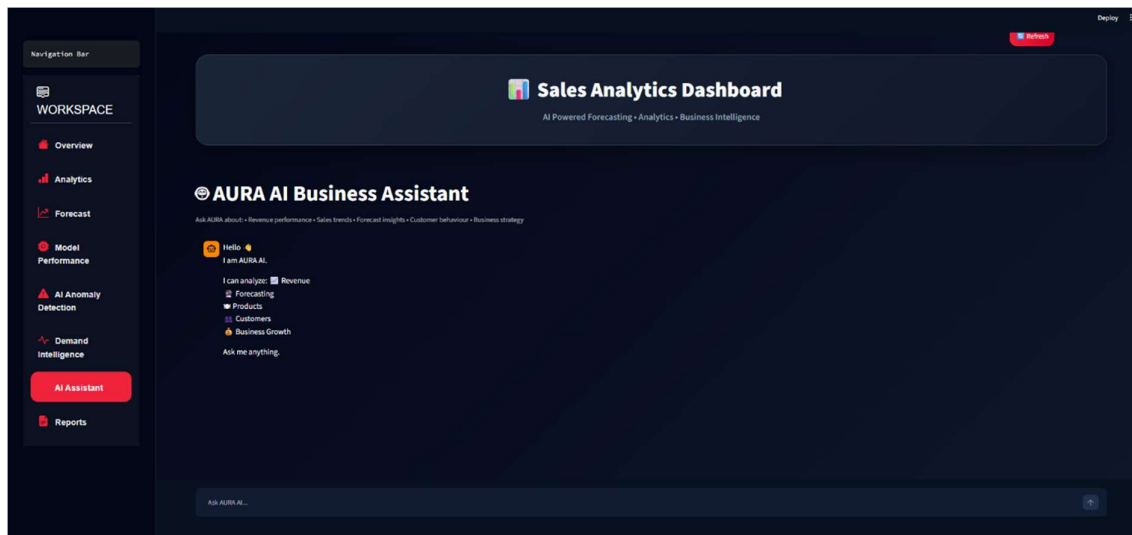
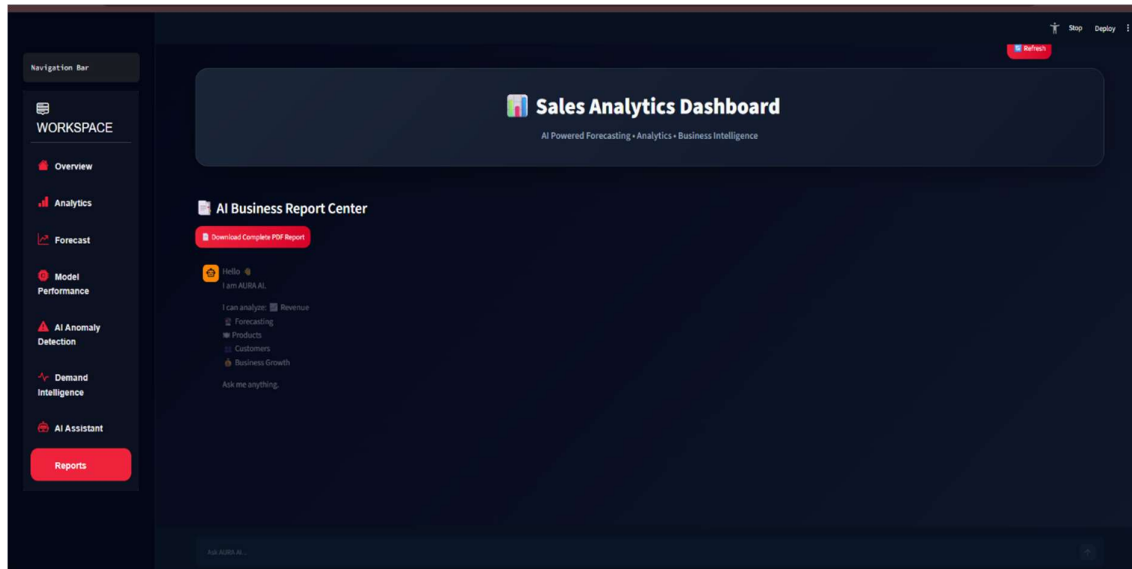
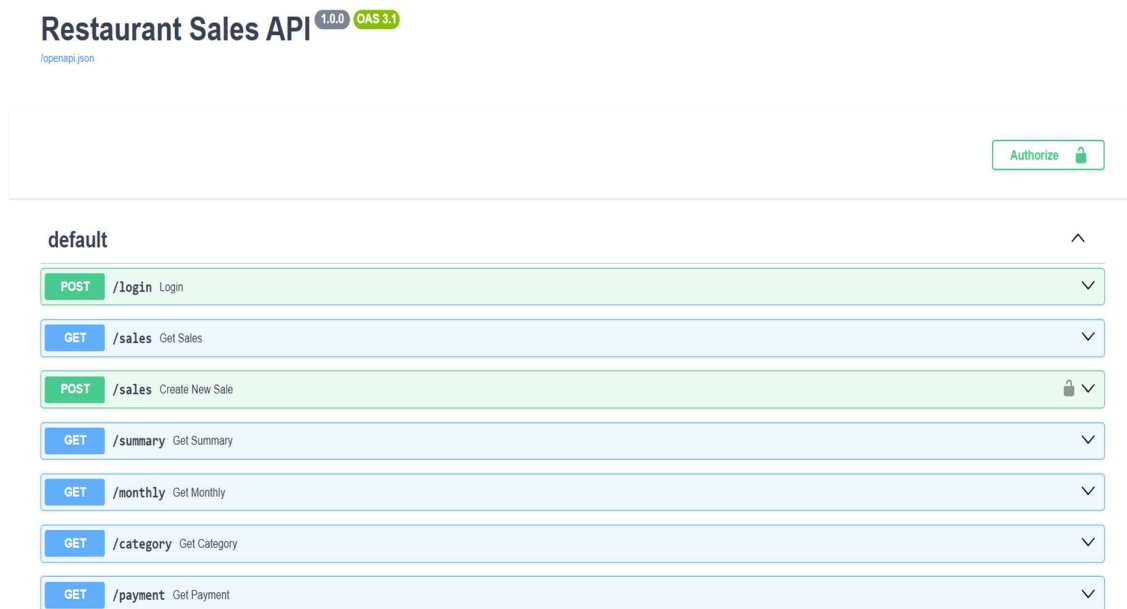


Figure F.9

Reports

**Figure F.9**

Swagger interface



GET	/top-products	Get Top Products	✓
POST	/upload-csv	Upload Csv	✓
POST	/upload-json	Upload Json	✓
POST	/approve	Approve	✓
GET	/sales/{bill_no}	Get Bill By Number	✓
PUT	/sales/{bill_no}	Update Existing Sale	🔒 ✓
DELETE	/sales/{bill_no}	Delete Sale	🔒 ✓
GET	/	Root	✓
POST	/sales/{bill_no}/restore	Restore Deleted Sale	🔒 ✓
Schemas			

References

- [1] S. Ramírez, *FastAPI Documentation*, 2024. Available: <https://fastapi.tiangolo.com/>
- [2] Streamlit Inc., *Streamlit Documentation*, 2024. Available: <https://docs.streamlit.io/>
- [3] PostgreSQL Global Development Group, *PostgreSQL Documentation*, 2024. Available: <https://www.postgresql.org/docs/>
- [4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [5] S. J. Taylor and B. Letham, "Forecasting at Scale," *PeerJ Computer Science*, vol. 3, e55, 2018.
- [6] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed., Morgan Kaufmann, 2012.
- [7] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O'Reilly Media, 2022.
- [8] SQLAlchemy Authors, *SQLAlchemy Documentation*, 2024. Available: <https://docs.sqlalchemy.org/>
- [9] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral Dissertation, University of California, Irvine, 2000.
- [10] Render Inc., *Render Documentation*, 2024. Available: <https://render.com/docs>

Thank You